

Audit: Rocq Bytecode Interpreter vs. OCaml 4.14 interp.c

Systematic Per-Instruction Comparison

Generated by audit infrastructure

March 26, 2026

Contents

1 Introduction

1.1 Purpose

This document provides a systematic, per-instruction audit of the formally verified OCaml bytecode interpreter written in Rocq (`Interp.v`) against two authoritative references:

1. **OCaml 4.14.2 runtime/interp.c** — the production C implementation of the ZINC bytecode interpreter.
2. **The Cadmium bytecode instruction specification** (`caml-instructions.pdf`) — a detailed reference manual for each instruction’s semantics.

The interpreter is the core *trusted* component of the verified compiler project. Any discrepancy between it and the real OCaml runtime constitutes a soundness risk for the entire verification effort.

1.2 Methodology

For each of the 107 instruction variants defined in `AST.v`, we:

1. Extract the exact C code from `interp.c` (OCaml 4.14.2).
2. Extract the exact Rocq code from `Interp.v`.
3. Paraphrase the PDF specification.
4. Compare the three, noting any discrepancies.
5. Assign a *status* (**OK**, bug, or needs-test) and a *risk level* (low, medium, or high).

Code snippets are auto-extracted by `extract_snippets.py` so the document stays current as `Interp.v` evolves.

1.3 Scope

- **In scope:** All 107 instruction variants in `AST.v`, the machine state model (`Machine.v`), the value representation (`Value.v`), and helper functions (`do_raise`, `z_flip_sign`, `field_or_heap`, etc.).
- **Out of scope:** GC interactions, threading, signal handling (modeled as no-ops), native code compilation, and debugger integration.

1.4 Key Architectural Differences

Aspect	OCaml <code>interp.c</code>	Rocq <code>Interp.v</code>
Stack	C pointer <code>sp</code> (grows down)	Functional list (grows with cons)
Heap	GC-managed C heap	<code>PositiveMap</code> (immutable map)
Values	Tagged words (tag bit)	Algebraic type (<code>Val_int</code> , <code>Val_block</code> , etc.)
Closures	Pointer + infix offset via pointer arithmetic	<code>Val_closure</code> <code>addr ofs</code>
PC	C <code>code_t</code> pointer	Z index into <code>PrimArray</code>
Trap pointer	C pointer <code>Caml_state->trapsp</code>	<code>trap_sp : nat = stack depth</code>

Aspect	OCaml <code>interp.c</code>	Rocq <code>Interp.v</code>
C calls	Direct function pointer call	<code>CCall_request</code> step result
GC	Triggered by allocation	No GC (immutable heap)
Signals	<code>caml_something_to_do</code>	Ignored (no-op)
Word size	63-bit integers (tagged)	Arbitrary-precision Z (extraction uses 63-bit)

2 Machine Model

2.1 State Record

The Rocq machine state (`Machine.v`):

```

1 Record state : Type      mk_state {
2   pc          : Z;
3   accu        : value;
4   stack       : list value;
5   env         : value;
6   extra_args  : nat;
7   global      : list value;
8   trap_sp     : nat;
9   hp          : heap;
10  next_addr   : nat;
11 }.

```

This corresponds to the C registers:

```

1 register code_t pc;
2 register value * sp;      // our stack : list value
3 register value accu;
4 value env;
5 intnat extra_args;
6 // Caml_state->trapsp      // our trap_sp : nat
7 // caml_global_data        // our global : list value

```

Analysis. The correspondence is direct. The main difference is that the C `sp` is a downward-growing pointer while our `stack` is an upward-growing cons-list. Both represent the same abstract stack. The `trap_sp` field stores the stack *depth* at the time of the last `PUSHTRAP`, while `interp.c` stores an absolute pointer. The Rocq version must compute the offset via `length(stack) - trap_sp` to find the trap frame, which is equivalent.

2.2 Heap Model

The heap is a `PositiveMap.t (nat * list value)`, mapping addresses to (tag, fields) pairs. Allocation returns monotonically increasing addresses. There is no garbage collection; the heap only grows.

2.3 Value Representation

```

1 Inductive value : Type
2   | Val_int  : Z -> value
3   | Val_block : nat -> list value -> value
4   | Val_ptr  : nat -> value
5   | Val_closure : nat -> nat -> value.

```

In `interp.c`, the tag bit distinguishes integers from pointers. Our algebraic representation is unambiguous by construction. The `Val_closure` constructor explicitly tracks the base address and infix offset within a shared closure block, corresponding to the C pointer arithmetic `env + offset * sizeof(value)`.

3 Stack Manipulation

3.1 ACC

PDF Spec. `ACC n`: Sets `accu` to `sp[n]`. The specialized variants `ACC0`–`ACC7` are optimized forms.

```
359 | Instruct(ACC):
360 |     accu = sp[*pc++];
361 |     Next;
```

```
110 | | ACC n ⇒
111 |     match nth_error s.(stack) n with
112 |     | Some v ⇒ Step (st s pc' v s.(stack) s.(env) s.(extra_args)
113 |                     s.(global) s.(trap_sp))
113 |     | None ⇒ Error "ACC: stack underflow"
```

Analysis. Direct correspondence. The C `sp[n]` maps to `nth_error stack n`. Our version returns `Error` on out-of-bounds; the C version has undefined behavior (buffer overread).

Status. **OK**

Risk. **Low**

3.2 PUSH

PDF Spec. `PUSH`: Pushes `accu` onto the stack (`*--sp = accu`). Same as `PUSHACC0`.

```
339 | Instruct(PUSH): Instruct(PUSHACC0):
340 |     *--sp = accu; Next;
```

```
116 | | PUSH ⇒
117 |     Step (st s pc' s.(accu) (s.(accu) :: s.(stack)) s.(env)
117 |           s.(extra_args) s.(global) s.(trap_sp))
```

Analysis. `PUSH` conses `accu` onto the stack. Exact match.

Status. **OK**

Risk. **Low**

3.3 PUSHACC

PDF Spec. PUSHACC n : Push `accu`, then set `accu = sp[n]` (where n counts from the *new* stack top). Specialized forms PUSHACC1–PUSHACC7.

```
356 | Instruct(PUSHACC):  
357 |     *--sp = accu;  
358 |     /* Fallthrough */
```

```
119 | | PUSHACC n ⇒  
120 |     let new_stack = s.(accu) :: s.(stack) in  
121 |     match nth_error new_stack n with  
122 |     | Some v ⇒ Step (st s pc' v new_stack s.(env) s.(extra_args)  
123 |                   | None ⇒ Error "PUSHACC: stack underflow")  
123 |     s.(global) s.(trap_sp))
```

Analysis. In the C code, `*--sp = accu` pushes first, then `accu = sp[n]` reads from the new stack. Our Rocq code constructs `new_stack := accu :: stack` then does `nth_error new_stack n`, which is the same indexing. Exact match.

Status. OK

Risk. Low

3.4 POP

PDF Spec. POP n : Pops n values from the stack (`sp += n`).

```
363 | Instruct(POP):  
364 |     sp += *pc++;  
365 |     Next;
```

```
126 | | POP n ⇒  
127 |     Step (st s pc' s.(accu) (skipn n s.(stack)) s.(env)  
127 |           s.(extra_args) s.(global) s.(trap_sp))
```

Analysis. `sp += n` corresponds to `skipn n stack`. Exact match.

Status. OK

Risk. Low

3.5 ASSIGN

PDF Spec. ASSIGN n : Sets `sp[n] = accu`, then `accu = ()`.

```

366   Instruct(ASSIGN):
367       sp[*pc++] = accu;
368       accu = Val_unit;
369       Next;

129   | ASSIGN n ⇒
130       match set_nth s.(stack) n s.(accu) with
131       | Some new_stack ⇒ Step (st s pc' val_unit new_stack s.(env)
132                               s.(extra_args) s.(global) s.(trap_sp))
132       | None ⇒ Error "ASSIGN: stack underflow"

```

Analysis. The C code does `sp[*pc++] = accu; accu = Val_unit`. Our code uses `set_nth` stack n `accu`, which is a functional update returning `Some new_stack` on success. The error case covers out-of-bounds which is undefined behavior in C. Exact match on valid inputs.

Status. OK

Risk. Low

4 Environment Access

4.1 ENVACC

PDF Spec. ENVACC n : Sets `accu = Field(env, n)`. Specialized variants ENVACC1–ENVACC4.

```

394   Instruct(ENVACC):
395       accu = Field(env, *pc++);
396       Next;

135   | ENVACC n ⇒
136       match field_or_heap s s.(env) n with
137       | Some v ⇒ Step (st s pc' v s.(stack) s.(env) s.(extra_args)
138                       s.(global) s.(trap_sp))
138       | None ⇒ Error "ENVACC: env access out of bounds"

```

Analysis. `Field(env, n)` corresponds to `field_or_heap s env n`. The `field_or_heap` helper handles both inline `Val_block` and heap-allocated `Val_ptr/Val_closure` uniformly, accounting for the offset in closures. Exact match.

Status. OK

Risk. Low

4.2 PUSHENVACC

PDF Spec. PUSHENVACC n : Push `accu`, then set `accu = Field(env, n)`.

```

391 | Instruct(PUSHENVACC):
392 |     --sp = accu;
393 |     /* Fallthrough */

```

```

141 | | PUSHENVACC n ⇒
142 |     let new_stack      s.(accu) :: s.(stack) in
143 |     match field_or_heap s s.(env) n with
144 |     | Some v ⇒ Step (st s pc' v new_stack s.(env) s.(extra_args)
145 |                     s.(global) s.(trap_sp))
146 |     | None ⇒ Error "PUSHENVACC: env access out of bounds"
147 |
148 |     (* PUSH_RETADDR: pushes [ret_addr, env, extra_args] onto the stack.
149 |       In interp.c: sp[0]=pc+ofs, sp[1]=env,
150 |       sp[2]=Long_val(extra_args). *)

```

Analysis. Same pattern as PUSH + ENVACC. Exact match.

Status. OK

Risk. Low

4.3 OFFSETCLOSURE

PDF Spec. OFFSETCLOSURE n : Sets $\text{accu} = \text{env} + n * \text{sizeof}(\text{value})$. This navigates within a flat closure block to access a different function in a mutually recursive group. Specialized: OFFSETCLOSURE0 ($n = 0$), OFFSETCLOSURE3 ($n = 3$), OFFSETCLOSUREM3 ($n = -3$).

```

615 | Instruct(OFFSETCLOSURE):
616 |     accu = env + *pc++ * sizeof(value); Next;

```

```

362 | | OFFSETCLOSURE ofs ⇒
363 |     match s.(env) with
364 |     | Val_closure addr base_ofs ⇒
365 |         let new_ofs      Z.to_nat (Z.of_nat base_ofs + ofs) in
366 |         Step (st s pc' (Val_closure addr new_ofs) s.(stack) s.(env)
367 |               s.(extra_args) s.(global) s.(trap_sp))
368 |     | Val_block t _ ⇒
369 |         if Z.eqb ofs 0 then
370 |             Step (st s pc' s.(env) s.(stack) s.(env) s.(extra_args)
371 |                   s.(global) s.(trap_sp))
372 |         else Error "OFFSETCLOSURE: non-zero offset on non-closure env"
373 |     | _ ⇒ Error "OFFSETCLOSURE: invalid env"

```

Analysis. The C code uses pointer arithmetic: $\text{accu} = \text{env} + *pc * \text{sizeof}(\text{value})$. Our code adjusts the offset within a Val_closure: the new offset is $\text{base_ofs} + \text{ofs}$. The multiplication by $\text{sizeof}(\text{value})$ is implicit because our offsets count fields, not bytes.

For Val_block with offset 0, we return env itself, which is correct for the self-reference case (OFFSETCLOSURE0). Non-zero offsets on non-closure values produce an error, which is safe since well-formed bytecode always has a closure in env when OFFSETCLOSURE is executed.

Status. **OK**

Risk. **Low**

4.4 PUSHOFFSETCLOSURE

PDF Spec. PUSHOFFSETCLOSURE n : Push `accu`, then OFFSETCLOSURE n .

```
613 | Instruct(PUSHOFFSETCLOSURE):
614 |     |--sp = accu; /* fallthrough */

374 | | PUSHOFFSETCLOSURE ofs =>
375 | |   let new_stack = s.(accu) :: s.(stack) in
376 | |   match s.(env) with
377 | |   | Val_closure addr base_ofs =>
378 | |     let new_ofs = Z.to_nat (Z.of_nat base_ofs + ofs) in
379 | |     Step (st s pc' (Val_closure addr new_ofs) new_stack s.(env)
380 | |           s.(extra_args) s.(global) s.(trap_sp))
381 | |   | Val_block t _ =>
382 | |     if Z.eqb ofs 0 then
383 | |       Step (st s pc' s.(env) new_stack s.(env) s.(extra_args)
384 | |             s.(global) s.(trap_sp))
385 | |     else Error "PUSHOFFSETCLOSURE: non-zero offset on non-closure
386 | |               env"
387 | |   | _ => Error "PUSHOFFSETCLOSURE: invalid env"
```

Analysis. Same as PUSH + OFFSETCLOSURE. Exact match.

Status. **OK**

Risk. **Low**

5 Function Application

5.1 PUSH_RETADDR

PDF Spec. Pushes a return frame [`pc+ofs`, `env`, `extra_args`] onto the stack, preparing for a multi-argument function call.

```
400 | Instruct(PUSH_RETADDR): {
401 |   sp -= 3;
402 |   sp[0] = (value) (pc + *pc);
403 |   sp[1] = env;
404 |   sp[2] = Val_long(extra_args);
405 |   pc++;
406 |   Next;
```



```

150 | PUSH_RETADDR ret_addr ⇒
151   let frame      Val_int ret_addr :: s.(env) :: Val_int (Z.of_nat
    s.(extra_args)) :: s.(stack) in
152   Step (st s pc' s.(accu) frame s.(env) s.(extra_args) s.(global)
    s.(trap_sp))
153
154   (* APPLY n: tail call with n args already on stack; sets
    extra_args = n-1. *)

```

Analysis. The C code pushes $sp[0]=(value)(pc+*pc)$, $sp[1]=env$, $sp[2]=Val_long(extra_args)$. Our code pushes $Val_int\ ret_addr :: env :: Val_int(Z.of_nat\ extra_args) :: stack$. The return address is pre-decoded to an absolute offset in our model (the loader resolves relative offsets), matching the C $pc + *pc$. Exact match.

Status. OK

Risk. Low

5.2 APPLY

PDF Spec. $APPLY\ n$: Direct call. Sets $extra_args = n - 1$, $pc = Code_val(accu)$, $env = accu$. The n arguments are already on the stack (pushed by $PUSH_RETADDR$ sequence).

```

408   Instruct(APPLY): {
409     extra_args = *pc - 1;
410     pc = Code_val(accu);
411     env = accu;
412     goto check_stacks;
413   }

```

```

155 | APPLY n ⇒
156   match get_code_ptr_s s s.(accu) with
157   | Some target_pc ⇒
158     Step (st s target_pc s.(accu) s.(stack) s.(accu) (Nat.sub n 1)
    s.(global) s.(trap_sp))
159   | None ⇒ Error "APPLY: accu is not a closure"
160
161   (* APPLY1/2/3: save return frame then call closure. *)

```

Analysis. The C code does $extra_args = *pc - 1$; $pc = Code_val(accu)$; $env = accu$. Our code extracts the code pointer via $get_code_ptr_s$ and sets $env = accu$ (the closure itself becomes the environment). $extra_args$ is set to $n - 1$. Exact match.

The C code also does $goto\ check_stacks$ for potential stack overflow reallocation, which we skip (no stack overflow in our model).

Status. OK

Risk. Low

5.3 APPLY1

PDF Spec. APPLY1: Call with 1 argument. Saves return frame inline: [arg1, pc, env, extra_args], then jumps to closure.

```
414   Instruct(APPLY1): {
415       value arg1 = sp[0];
416       sp -= 3;
417       sp[0] = arg1;
418       sp[1] = (value)pc;
419       sp[2] = env;
420       sp[3] = Val_long(extra_args);
421       pc = Code_val(accum);
422       env = accum;
423       extra_args = 0;
424       goto check_stacks;
425   }
```

```
163   | APPLY1 ⇒
164       match s.(stack) with
165   | arg1 :: rest ⇒
166       match get_code_ptr_s s s.(accum) with
167   | Some target_pc ⇒
168       let new_stack      arg1 :: Val_int pc' :: s.(env) :: Val_int
169         (Z.of_nat s.(extra_args)) :: rest in
170       Step (st s target_pc s.(accum) new_stack s.(accum) 0
171            s.(global) s.(trap_sp))
171   | None ⇒ Error "APPLY1: accum is not a closure"
171   | _   ⇒ Error "APPLY1: stack underflow"
```

Analysis. C: sp[0]=arg1, sp[1]=pc, sp[2]=env, sp[3]=Val_long(extra_args). Rocq: arg1 :: Val_int pc' :: env :: Val_int(extra_args) :: rest. Exact match. extra_args = 0 in both cases.

Status. OK

Risk. Low

5.4 APPLY2

PDF Spec. APPLY2: Call with 2 arguments.

```
426   Instruct(APPLY2): {
427       value arg1 = sp[0];
428       value arg2 = sp[1];
429       sp -= 3;
430       sp[0] = arg1;
431       sp[1] = arg2;
432       sp[2] = (value)pc;
433       sp[3] = env;
434       sp[4] = Val_long(extra_args);
435       pc = Code_val(accum);
```

```

436     env = accu;
437     extra_args = 1;
438     goto check_stacks;
439 }

```

```

175 | APPLY2 ⇒
176   match s.(stack) with
177   | arg1 :: arg2 :: rest ⇒
178     match get_code_ptr_s s s.(accu) with
179     | Some target_pc ⇒
180       let new_stack    arg1 :: arg2 :: Val_int pc' :: s.(env) ::
181         Val_int (Z.of_nat s.(extra_args)) :: rest in
182       Step (st s target_pc s.(accu) new_stack s.(accu) 1
183             s.(global) s.(trap_sp))
182   | None ⇒ Error "APPLY2: accu is not a closure"
183   | _   ⇒ Error "APPLY2: stack underflow"

```

Analysis. Same pattern as APPLY1 with 2 args. `extra_args = 1`. Exact match.

Status. **OK**

Risk. **Low**

5.5 APPLY3

PDF Spec. APPLY3: Call with 3 arguments.

```

440   Instruct(APPLY3): {
441     value arg1 = sp[0];
442     value arg2 = sp[1];
443     value arg3 = sp[2];
444     sp -= 3;
445     sp[0] = arg1;
446     sp[1] = arg2;
447     sp[2] = arg3;
448     sp[3] = (value)pc;
449     sp[4] = env;
450     sp[5] = Val_long(extra_args);
451     pc = Code_val(accu);
452     env = accu;
453     extra_args = 2;
454     goto check_stacks;
455   }

```

```

187 | APPLY3 ⇒
188   match s.(stack) with
189   | arg1 :: arg2 :: arg3 :: rest ⇒
190     match get_code_ptr_s s s.(accu) with
191     | Some target_pc ⇒
192       let new_stack    arg1 :: arg2 :: arg3 :: Val_int pc' ::
193         s.(env) :: Val_int (Z.of_nat s.(extra_args)) :: rest in

```

```

193     Step (st s target_pc s.(accu) new_stack s.(accu) 2
        s.(global) s.(trap_sp))
194   | None ⇒ Error "APPLY3: accu is not a closure"
195   | _   ⇒ Error "APPLY3: stack underflow"
196
197   (* APPTERM n s: slide top n args down by (slotsize - n), tail
        call. *)

```

Analysis. Same pattern with 3 args. `extra_args = 2`. Exact match.

Status. OK

Risk. Low

5.6 APPTERM

PDF Spec. `APPTERM n s`: Tail call. Slides the top n stack words down over a frame of size s , then jumps to the closure in `accu`. `extra_args += n - 1`.

```

457   Instruct(APPTERM): {
458     int nargs = *pc++;
459     int slotsize = *pc;
460     value * newsp;
461     int i;
462     /* Slide the nargs bottom words of the current frame to the top
463        of the frame, and discard the remainder of the frame */
464     newsp = sp + slotsize - nargs;
465     for (i = nargs - 1; i >= 0; i--) newsp[i] = sp[i];
466     sp = newsp;
467     pc = Code_val(accu);
468     env = accu;
469     extra_args += nargs - 1;
470     goto check_stacks;
471   }

```

```

200   | APPTERM nargs slotsize ⇒
201     let args    firstn nargs s.(stack) in
202     let base    skipn slotsize s.(stack) in
203     match get_code_ptr_s s s.(accu) with
204     | Some target_pc ⇒
205       Step (st s target_pc s.(accu) (args ++ base) s.(accu) (Nat.add
        s.(extra_args) (Nat.sub nargs 1)) s.(global) s.(trap_sp))
206     | None ⇒ Error "APPTERM: accu is not a closure"

```

Analysis. C: slides `sp[0..n-1]` to `sp+slotsize-n` via loop. Rocq: `firstn nargs stack ++ skipn slotsize stack`. Both produce the same result: the top n values above the discarded $slotsize - n$ values. Exact match.

Status. OK

Risk. **Low**

5.7 APPTERM1

PDF Spec. APPTERM1 *s*: Tail call with 1 argument.

```
472   Instruct(APPTERM1): {  
473     value arg1 = sp[0];  
474     sp = sp + *pc - 1;  
475     sp[0] = arg1;  
476     pc = Code_val(accum);  
477     env = accum;  
478     goto check_stacks;  
479   }
```

```
209   | APPTERM1 slotsize =>  
210     match s.(stack) with  
211     | arg1 :: _ =>  
212       let base      skipn slotsize s.(stack) in  
213       match get_code_ptr_s s s.(accum) with  
214       | Some target_pc =>  
215         Step (st s target_pc s.(accum) (arg1 :: base) s.(accum)  
              s.(extra_args) s.(global) s.(trap_sp))  
216       | None => Error "APPTERM1: accum is not a closure"  
217     | _ => Error "APPTERM1: stack underflow"
```

Analysis. C: $sp = sp + *pc - 1$; $sp[0] = arg1$. Rocq: $arg1 :: skipn\ slotsize\ stack$. Both keep 1 arg and discard the frame. *extra_args* unchanged. Exact match.

Status. **OK**

Risk. **Low**

5.8 APPTERM2

PDF Spec. APPTERM2 *s*: Tail call with 2 arguments.

```
480   Instruct(APPTERM2): {  
481     value arg1 = sp[0];  
482     value arg2 = sp[1];  
483     sp = sp + *pc - 2;  
484     sp[0] = arg1;  
485     sp[1] = arg2;  
486     pc = Code_val(accum);  
487     env = accum;  
488     extra_args += 1;  
489     goto check_stacks;  
490   }
```

```

221 | APPTERM2 slotsize ⇒
222   match s.(stack) with
223   | arg1 :: arg2 :: _ ⇒
224     let base      skipn slotsize s.(stack) in
225     match get_code_ptr_s s s.(accu) with
226     | Some target_pc ⇒
227       Step (st s target_pc s.(accu) (arg1 :: arg2 :: base)
             s.(accu) (Nat.add s.(extra_args) 1) s.(global)
             s.(trap_sp))
228     | None ⇒ Error "APPTERM2: accu is not a closure"
229     | _   ⇒ Error "APPTERM2: stack underflow"

```

Analysis. Same pattern, 2 args. `extra_args += 1`. Exact match.

Status. OK

Risk. Low

5.9 APPTERM3

PDF Spec. APPTERM3 *s*: Tail call with 3 arguments.

```

491 Instruct(APPTERM3): {
492   value arg1 = sp[0];
493   value arg2 = sp[1];
494   value arg3 = sp[2];
495   sp = sp + *pc - 3;
496   sp[0] = arg1;
497   sp[1] = arg2;
498   sp[2] = arg3;
499   pc = Code_val(accu);
500   env = accu;
501   extra_args += 2;
502   goto check_stacks;
503 }

```

```

233 | APPTERM3 slotsize ⇒
234   match s.(stack) with
235   | arg1 :: arg2 :: arg3 :: _ ⇒
236     let base      skipn slotsize s.(stack) in
237     match get_code_ptr_s s s.(accu) with
238     | Some target_pc ⇒
239       Step (st s target_pc s.(accu) (arg1 :: arg2 :: arg3 :: base)
             s.(accu) (Nat.add s.(extra_args) 2) s.(global)
             s.(trap_sp))
240     | None ⇒ Error "APPTERM3: accu is not a closure"
241     | _   ⇒ Error "APPTERM3: stack underflow"
242
243   (* RETURN n: pop n locals; if extra_args > 0 tail-call accu, else
      restore return frame. *)

```

Analysis. Same pattern, 3 args. `extra_args += 2`. Exact match.

Status. **OK**

Risk. **Low**

5.10 RETURN

PDF Spec. RETURN n : Pop n locals. If `extra_args > 0`, decrement and tail-call `accu`; otherwise restore the return frame [`pc`, `env`, `extra_args`].

```
505   Instruct(RETURN): {
506     sp += *pc++;
507     if (extra_args > 0) {
508       extra_args--;
509       pc = Code_val(accu);
510       env = accu;
511     } else {
512       pc = (code_t)(sp[0]);
513       env = sp[1];
514       extra_args = Long_val(sp[2]);
515       sp += 3;
516     }
517     Next;
```

```
246 | RETURN stacksize =>
247   let stk      skipn stacksize s.(stack) in
248   if Nat.ltb 0 s.(extra_args) then
249     match get_code_ptr_s s s.(accu) with
250     | Some target_pc =>
251       Step (st s target_pc s.(accu) stk s.(accu) (Nat.sub
252         s.(extra_args) 1) s.(global) s.(trap_sp))
253     | None => Error "RETURN: accu is not a closure"
254   else
255     match stk with
256     | Val_int ret_pc :: saved_env :: Val_int saved_ea :: rest =>
257       Step (st s ret_pc s.(accu) rest saved_env (Z.to_nat
258         saved_ea) s.(global) s.(trap_sp))
259     | _ => Error "RETURN: malformed return frame"
258
259   (* RESTART: restore args from partial application closure in env.
    *)
```

Analysis. C: `sp += *pc`; if (`extra_args > 0`) { `extra_args--`; `pc = Code_val(accu)`; `env = accu` } else { `pc = sp[0]`; `env = sp[1]`; `extra_args = Long_val(sp[2])`; `sp += 3` }.

Rocq: `skipn stacksize stack`, then checks `Nat.ltb 0 extra_args`. If true, decrements and calls closure. Otherwise, pattern-matches the return frame `Val_int ret_pc :: saved_env :: Val_int saved_ea :: rest`.

The C check is `extra_args > 0` while Rocq uses `Nat.ltb 0 extra_args`, which is equivalent. Exact match.

Status. **OK**

Risk. **Low**

6 Closure Creation

6.1 RESTART

PDF Spec. RESTART: Restores partially applied arguments from the current closure environment. The closure layout is [code, closinfo, saved_env, arg0, arg1, ...]. Extra args from field 3 onward are pushed back onto the stack, and env is restored from field 2.

```
520 Instruct(RESTART): {
521   int num_args = Wosize_val(env) - 3;
522   int i;
523   sp -= num_args;
524   for (i = 0; i < num_args; i++) sp[i] = Field(env, i + 3);
525   env = Field(env, 2);
526   extra_args += num_args;
527   Next;
```

```
262 | RESTART ⇒
263   let restart_fields (all_fields : list value) (ofs : nat) :
264     step_result
265     let fields      skipn ofs all_fields in
266     let num_args    Nat.sub (length fields) 3 in
267     let args        skipn 3 fields in
268     let new_stack   args ++ s.(stack) in
269     match nth_error fields 2 with
270     | Some saved_env ⇒
271       Step (st s pc' s.(accu) new_stack saved_env (Nat.add
272         s.(extra_args) num_args) s.(global) s.(trap_sp))
273     | None ⇒ Error "RESTART: malformed closure"
274     end in
275   match s.(env) with
276   | Val_closure addr ofs ⇒
277     match heap_lookup s.(hp) addr with
278     | Some (t, all_fields) ⇒
279       if Nat.eqb t Closure_tag then restart_fields all_fields ofs
280       else Error "RESTART: env is not a closure"
281     | None ⇒ Error "RESTART: dangling pointer"
282   | Val_block t fields ⇒
283     if Nat.eqb t Closure_tag then restart_fields fields 0%nat
284     else Error "RESTART: env is not a closure"
285   | _ ⇒ Error "RESTART: env is not a block"
286   (* GRAB n: if extra_args >= n, consume n and continue;
287     else build partial application closure and return to caller. *)
```

Analysis. C: num_args = Wosize_val(env) - 3; pushes Field(env, i+3) for $i = 0..n - 1$; sets env = Field(env, 2).

Rocq: for `Val_closure` `addr ofs`, reads the heap block, computes `fields := skipn ofs all_fields`, then `num_args = length(fields) - 3`, pushes `skipn 3 fields`, and sets `env` to `fields[2]`.

Key difference: the Rocq version handles the `Val_closure` offset correctly, accounting for infix closures within a shared block. The C version uses `Wosize_val(env)` which gives the total block size, and `env` already points to the right sub-closure. In the Rocq model, the offset adjusts the view. Functionally equivalent.

Status. OK

Risk. Medium— The closure offset arithmetic for mutually recursive functions needs careful testing.

6.2 GRAB

PDF Spec. GRAB n : If `extra_args >= n`, consume them and continue. Otherwise, build a partial application closure containing the saved arguments and return to the caller.

```

530   Instruct(GRAB): {
531       int required = *pc++;
532       if (extra_args >= required) {
533           extra_args -= required;
534       } else {
535           msize_t num_args, i;
536           num_args = 1 + extra_args; /* arg1 + extra args */
537           Alloc_small(accum, num_args + 3, Closure_tag);
538           Field(accum, 2) = env;
539           for (i = 0; i < num_args; i++) Field(accum, i + 3) = sp[i];
540           Code_val(accum) = pc - 3; /* Point to the preceding RESTART
                                     instr. */
541           Closinfo_val(accum) = Make_closinfo(0, 2);
542           sp += num_args;
543           pc = (code_t)(sp[0]);
544           env = sp[1];
545           extra_args = Long_val(sp[2]);
546           sp += 3;
547       }
548       Next;

```

```

289 | GRAB required ⇒
290   if Nat.leb required s.(extra_args) then
291       Step (st s pc' s.(accum) s.(stack) s.(env) (Nat.sub
                s.(extra_args) required) s.(global) s.(trap_sp))
292   else
293       let num_args      S s.(extra_args) in
294       let saved_args     firstn num_args s.(stack) in
295       let rest_stack     skipn num_args s.(stack) in
296       (* Partial application closure: [code=RESTART, closinfo,
                saved_env, arg1, ...] *)
297       let closinfo       Val_int 0 in
298       let fields         Val_int (s.(pc) - 1) :: closinfo :: s.(env) ::
                saved_args in
299       let '(s', base_ptr) heap_alloc s Closure_tag fields in

```

```

300   let addr      match base_ptr with Val_ptr a ⇒ a | _ ⇒ 0%nat
      end in
301   let closure    Val_closure addr 0%nat in
302   match rest_stack with
303   | Val_int ret_pc :: saved_env :: Val_int saved_ea :: rest ⇒
304     Step (st s' ret_pc closure rest saved_env (Z.to_nat
      saved_ea) s.(global) s.(trap_sp))
305   | _ ⇒ Error "GRAB: malformed return frame"
306
307   (* CLOSURE n ofs: build closure of n+1 fields [code, closinfo, v0,
      ..., vn-1].
308   If n > 0 the accumulator is pushed first (it becomes v0). *)

```

Analysis. Sufficient args path: Both decrement `extra_args` by required and continue. Exact match.

Partial application path:

- C: Allocates closure of size `num_args + 3` with tag `Closure_tag`. Layout: `Code_val = pc - 3` (points to RESTART), `Closinfo_val = Make_closinfo(0, 2)`, `Field(2) = env`, `Field(3..) = args`.
- Rocq: Allocates with fields `Val_int(pc-1) :: closinfo :: env :: saved_args.closinfo = Val_int 0`.

Discrepancy: closinfo value. The C code uses `Make_closinfo(0, 2)` which encodes the arity and start-of-environment offset as a tagged value. Our code uses `Val_int 0`. This is **safe** as long as nothing in our model inspects the `closinfo` field — and currently nothing does. However, if we later need to introspect closure metadata (e.g., for marshaling or debugging), this will need to be fixed.

Discrepancy: code pointer offset. The C code points to `pc - 3` (the RESTART instruction, which is 3 words before GRAB). Our code uses `pc - 1`. This difference exists because the C instruction stream has the RESTART opcode at `pc-3` (accounting for the GRAB opcode word and its operand word and then the RESTART before that), while in our model, the RESTART instruction is at index `pc - 1` in the instruction array (the instruction before GRAB). Both correctly point to RESTART.

Status. NEEDS TEST: Verify `closinfo=0` does not cause issues with RESTART; verify `pc-1` correctly targets RESTART in all test programs.

Risk. High: The `closinfo` field is hardcoded to 0 instead of `Make_closinfo(0, 2)`. Safe currently but fragile.

6.3 CLOSURE

PDF Spec. CLOSURE *n ofs*: Build a closure with *n* environment variables and code pointer at `pc + ofs`. Layout: `[code_ptr, closinfo, v0, ..., v_n-1]`. If *n* > 0, push `accu` first (it becomes `v0`).

```

551   Instruct(CLOSURE): {
552     int nvars = *pc++;
553     int i;
554     if (nvars > 0) *--sp = accu;
555     if (nvars <= Max_young_wosize - 2) {

```

```

556      /* nvars + 2 <= Max_young_wosize, can allocate in minor heap
      */
557      Alloc_small(accu, 2 + nvars, Closure_tag);
558      for (i = 0; i < nvars; i++) Field(accu, i + 2) = sp[i];
559  } else {
560      /* PR#6385: must allocate in major heap */
561      /* caml_alloc_shr and caml_initialize never trigger a GC,
562         so no need to Setup_for_gc */
563      accu = caml_alloc_shr(2 + nvars, Closure_tag);
564      for (i = 0; i < nvars; i++) caml_initialize(&Field(accu, i +
565         2), sp[i]);
566  }
567  /* The code pointer is not in the heap, so no need to go
568     through
569     caml_initialize. */
570  Code_val(accu) = pc + *pc;
571  Closinfo_val(accu) = Make_closinfo(0, 2);
572  pc++;
573  sp += nvars;
574  Next;

```

```

310  | CLOSURE nvars code_ofs =>
311      let stk      if Nat.ltb 0 nvars then s.(accu) :: s.(stack) else
312                    s.(stack) in
313      let vars      firstn nvars stk in
314      let rest      skipn nvars stk in
315      let closinfo   Val_int 0 in
316      let fields     Val_int code_ofs :: closinfo :: vars in
317      let '(s', base_ptr) heap_alloc s Closure_tag fields in
318      let addr       match base_ptr with Val_ptr a => a | _ => 0%nat end
319                      in
320      let closure     Val_closure addr 0%nat in
321      Step (st s' pc' closure rest s.(env) s.(extra_args) s.(global)
322          s.(trap_sp))
323
324  (* CLOSUREREC nf nv [ofs0;ofs1;...]: build flat closure block of
325     (nf*3-1+nv) fields. Layout: [code0,ci0, infix,code1,ci1, ...,
326     v0,v1,...]
327     Each closure_i is pushed as Val_closure(addr, 3*i). *)

```

Analysis. Both conditionally push `accu` when `nvars > 0`, allocate a block of `2 + nvars` fields with `Closure_tag`, fill code pointer and environment variables, and set `closinfo`.

Same `closinfo` discrepancy as GRAB: we use `Val_int 0` vs. `Make_closinfo(0, 2)`.

The code offset is pre-resolved in our model (absolute), matching the C `pc + *pc` after loader processing.

Status. OK

Risk. Medium— `closinfo=0` (same systemic issue as GRAB).

6.4 CLOSUREREC

PDF Spec. `CLOSUREREC nf nv [ofs0, ..., ofsnf-1]`: Build a flat mutual recursion closure block. Total size is $3 \times nf - 1 + nv$. Layout: `[code0, ci0, infix1, code1, ci1, ..., v0, ..., vnv-1]`. Each function i is accessed at offset $3i$ from the block start. The closures are pushed onto the stack in order, with `accu` set to closure 0.

```

575 Instruct(CLOSUREREC): {
576   int nfuncs = *pc++;
577   int nvars = *pc++;
578   mlsize_t envofs = nfuncs * 3 - 1;
579   mlsize_t blksize = envofs + nvars;
580   int i;
581   value * p;
582   if (nvars > 0) *--sp = accu;
583   if (blksize <= Max_young_wosize) {
584     Alloc_small(accu, blksize, Closure_tag);
585     p = &Field(accu, envofs);
586     for (i = 0; i < nvars; i++, p++) *p = sp[i];
587   } else {
588     /* PR#6385: must allocate in major heap */
589     /* caml_alloc_shr and caml_initialize never trigger a GC,
590        so no need to Setup_for_gc */
591     accu = caml_alloc_shr(blksize, Closure_tag);
592     p = &Field(accu, envofs);
593     for (i = 0; i < nvars; i++, p++) caml_initialize(p, sp[i]);
594   }
595   sp += nvars;
596   /* The code pointers and infix headers are not in the heap,
597      so no need to go through caml_initialize. */
598   *--sp = accu;
599   p = &Field(accu, 0);
600   *p++ = (value) (pc + pc[0]);
601   *p++ = Make_closinfo(0, envofs);
602   for (i = 1; i < nfuncs; i++) {
603     *p++ = Make_header(i * 3, Infix_tag, Caml_white); /* color
604                                                        irrelevant */
605     *--sp = (value) p;
606     *p++ = (value) (pc + pc[i]);
607     envofs -= 3;
608     *p++ = Make_closinfo(0, envofs);
609   }
610   pc += nfuncs;
611   Next;

```

```

324 | CLOSUREREC nfuncs nvars code_offsets =>
325   let stk      if Nat.ltb 0 nvars then s.(accu) :: s.(stack) else
326               s.(stack) in
327   let vars     firstn nvars stk in
328   let rest     skipn nvars stk in
329   match code_offsets with
330   | [] => Error "CLOSUREREC: no code offsets"
331   | _ =>
332     let closinfo Val_int 0 in
333     let infix_hdr Val_block Infix_tag [] in

```

```

333   let fix build_closure_fields (i : nat) (offsets : list Z) :
      list value
334   match offsets with
335   | [] => vars
336   | ofs :: rest_ofs =>
337     if Nat.eqb i 0 then
338       Val_int ofs :: closinfo :: build_closure_fields 1%nat
        rest_ofs
339     else
340       infix_hdr :: Val_int ofs :: closinfo ::
        build_closure_fields (S i) rest_ofs
341   end in
342   let fields      build_closure_fields 0%nat code_offsets in
343   let '(s', base_ptr)  heap_alloc s Closure_tag fields in
344   let addr      match base_ptr with Val_ptr a => a | _ => 0%nat
      end in
345   let closure_at (i : nat) : value
346     if Nat.eqb i 0 then Val_closure addr 0%nat
347     else Val_closure addr (3 * i) in
348   (* Push closures: closure_{nf-1} on top, closure_0 at bottom.
349      After this instruction accu = closure_0. *)
350   let fix push_closures (i : nat) (stk : list value) : list
      value
351   match i with
352   | 0 => stk
353   | S i' =>
354     let stk'      push_closures i' stk in
355     closure_at i' :: stk'
356   end in
357   let new_stack      push_closures nfuncs rest in
358   Step (st s' pc' (closure_at 0%nat) new_stack s.(env)
      s.(extra_args) s.(global) s.(trap_sp))
359
360   (* OFFSETCLOSURE n: accu      env offset by n fields (within the
      same heap block). *)

```

Analysis. This is one of the most complex instructions and deserves careful attention.

Block layout:

- C: Block size = $nf \times 3 - 1 + nv$. Fields: code0, closinfo0, [infix_hdr, code_i, closinfo_i]_i=1..nf-1, v0, ..., v_nv-1.
- Rocq: build_closure_fields produces the same layout: first function gets code :: closinfo, subsequent get infix_hdr :: code :: closinfo, then environment variables.

Infix headers:

- C: Make_header(i*3, Infix_tag, Caml_white) — this is a real GC header word encoding the infix offset.
- Rocq: Val_block Infix_tag [] — an empty block tagged Infix_tag. This does **not** encode the offset.

This is a **potential issue** if anything inspects the infix header to compute offsets. In practice, the header is only used by the GC (to scan through the block), and our model has no GC. The

`Val_closure` `addr ofs` representation tracks offsets explicitly, so the infix header content is irrelevant for execution.

Closinfo: Same systemic issue: `Val_int 0` vs. `Make_closinfo(0, envofs)`. The C version stores decreasing `envofs` values for each sub-closure; we store 0.

Push order: C pushes `accu` first (before allocation), then pushes sub-closures `1..nf-1` during field initialization. Rocq pushes via `push_closures` after allocation. Both result in closure `nf-1` on top and closure 0 in `accu`. The push function is recursive with `closure_at i'` for $i' = 0..nf-1$, pushing them bottom-up.

Status. **NEEDS TEST:** Verify push order matches for $nf > 2$; verify infix header irrelevance.

Risk. **High:** Complex instruction with three systemic discrepancies (infix headers, `closinfo`, push ordering). Covered by PBT but needs scrutiny.

7 Global Variables

7.1 GETGLOBAL

PDF Spec. `GETGLOBAL n`: Sets `accu = Field(global_data, n)`.

```
637   Instruct(GETGLOBAL):
638       accu = Field(caml_global_data, *pc);
639       pc++;
640       Next;
```

```
387   | GETGLOBAL n =>
388       match nth_error s.(global) n with
389       | Some v => Step (st s pc' v s.(stack) s.(env) s.(extra_args)
390                       s.(global) s.(trap_sp))
390       | None => Error "GETGLOBAL: index out of bounds"
```

Analysis. C: `Field(caml_global_data, *pc)`. Rocq: `nth_error global n`. Direct correspondence.

Status. **OK**

Risk. **Low**

7.2 PUSHGETGLOBAL

PDF Spec. `PUSHGETGLOBAL n`: Push `accu`, then `GETGLOBAL n`.

```
634   Instruct(PUSHGETGLOBAL):
635       *--sp = accu;
636       /* Fallthrough */
```

```

393 | PUSHGETGLOBAL n ⇒
394   let new_stack    s.(accu) :: s.(stack) in
395   match nth_error s.(global) n with
396   | Some v ⇒ Step (st s pc' v new_stack s.(env) s.(extra_args)
397                   s.(global) s.(trap_sp))
   | None ⇒ Error "PUSHGETGLOBAL: index out of bounds"

```

Analysis. Push + getglobal. Exact match.

Status. OK

Risk. Low

7.3 GETGLOBALFIELD

PDF Spec. GETGLOBALFIELD $n\ p$: $\text{accu} = \text{Field}(\text{Field}(\text{global_data}, n), p)$.

```

645 Instruct(GETGLOBALFIELD): {
646   accu = Field(caml_global_data, *pc);
647   pc++;
648   accu = Field(accu, *pc);
649   pc++;
650   Next;

```

```

400 | GETGLOBALFIELD n p ⇒
401   match nth_error s.(global) n with
402   | Some glob ⇒
403     match field_or_heap s glob p with
404     | Some v ⇒ Step (st s pc' v s.(stack) s.(env) s.(extra_args)
405                     s.(global) s.(trap_sp))
406     | None ⇒ Error "GETGLOBALFIELD: field access failed"
   | None ⇒ Error "GETGLOBALFIELD: index out of bounds"

```

Analysis. Two-level field access. Rocq uses `nth_error global n` then `field_or_heap` for field p . Exact match.

Status. OK

Risk. Low

7.4 PUSHGETGLOBALFIELD

PDF Spec. Push `accu`, then GETGLOBALFIELD.

```

642 Instruct(PUSHGETGLOBALFIELD):
643   *--sp = accu;
644   /* Fallthrough */

```

```

410 | PUSHGETGLOBALFIELD n p ⇒
411   let new_stack      s.(accu) :: s.(stack) in
412   match nth_error s.(global) n with
413   | Some glob ⇒
414     match field_or_heap s glob p with
415     | Some v ⇒ Step (st s pc' v new_stack s.(env) s.(extra_args)
416                     s.(global) s.(trap_sp))
416     | None ⇒ Error "PUSHGETGLOBALFIELD: field access failed"
417   | None ⇒ Error "PUSHGETGLOBALFIELD: index out of bounds"

```

Analysis. Same as push + getglobalfield. Exact match.

Status. OK

Risk. Low

7.5 SETGLOBAL

PDF Spec. SETGLOBAL n : Field(global_data, n) = accu; accu = ().

```

653   Instruct(SETGLOBAL):
654     caml_modify(&Field(caml_global_data, *pc), accu);
655     accu = Val_unit;
656     pc++;
657     Next;

```

```

421 | SETGLOBAL n ⇒
422   let new_global      match set_nth s.(global) n s.(accu) with
423                       | Some g ⇒ g | None ⇒ s.(global) end in
424   Step (st s pc' val_unit s.(stack) s.(env) s.(extra_args)
         new_global s.(trap_sp))

```

Analysis. C: caml_modify(&Field(caml_global_data, *pc), accu); accu = Val_unit. Rocq: set_nth global n accu, falls back to unchanged on error (silently). The silent fallback is slightly different from C (which would segfault on out-of-bounds), but both cases are unreachable with well-formed bytecode.

Status. OK

Risk. Low

8 Allocation

8.1 ATOM

PDF Spec. ATOM t : Sets accu to the atom with tag t (a zero-size block).


```

670 | Instruct(ATOM):
671 |     accu = Atom(*pc++); Next;

```

```

426 | | ATOM t ⇒
427 |     let '(s', ptr)      heap_alloc s t [] in
428 |     Step (st s' pc' ptr s.(stack) s.(env) s.(extra_args) s.(global)
        s.(trap_sp))

```

Analysis. C: `Atom(*pc++)` returns a pointer to a pre-allocated zero-size block. Rocq: `heap_alloc s t []` allocates a fresh empty block. Semantically equivalent—both produce a zero-size block with the given tag. The Rocq version allocates a new block each time (no sharing), which is safe since atoms are immutable.

Status. OK

Risk. Low

8.2 PUSHATOM

PDF Spec. Push `accu`, then `ATOM`.

```

667 | Instruct(PUSHATOM):
668 |     *--sp = accu;
669 |     /* Fallthrough */

```

```

430 | | PUSHATOM t ⇒
431 |     let new_stack      s.(accu) :: s.(stack) in
432 |     let '(s', ptr)      heap_alloc s t [] in
433 |     Step (st s' pc' ptr new_stack s.(env) s.(extra_args) s.(global)
        s.(trap_sp))
434 |
435 |     (* MAKEBLOCK tag size: accu=field0, pop (size-1) from stack. *)

```

Analysis. Push + atom. Exact match.

Status. OK

Risk. Low

8.3 MAKEBLOCK

PDF Spec. `MAKEBLOCK s t`: Allocate a block of size `s` with tag `t`. `Field(0)` = `accu`, remaining fields popped from stack.

```

673   Instruct(MAKEBLOCK): {
674       msize_t wosize = *pc++;
675       tag_t tag = *pc++;
676       msize_t i;
677       value block;
678       if (wosize <= Max_young_wosize) {
679           Alloc_small(block, wosize, tag);
680           Field(block, 0) = accu;
681           for (i = 1; i < wosize; i++) Field(block, i) = *sp++;
682       } else {
683           block = caml_alloc_shr(wosize, tag);
684           caml_initialize(&Field(block, 0), accu);
685           for (i = 1; i < wosize; i++) caml_initialize(&Field(block,
686               i), *sp++);
687       }
687       accu = block;
688       Next;

```

```

436 | MAKEBLOCK t size =>
437   let fields      s.(accu) :: firstn (Nat.sub size 1) s.(stack) in
438   let new_stack    skipn (Nat.sub size 1) s.(stack) in
439   let '(s', ptr)   heap_alloc s t fields in
440   Step (st s' pc' ptr new_stack s.(env) s.(extra_args) s.(global)
        s.(trap_sp))

```

Analysis. C: `Field(block, 0) = accu; for (i=1; i<wosize; i++) Field(block,i) = *sp++`.
Rocq: `accu :: firstn (size-1) stack`, then `heap_alloc`.

Note the C operand order is `wosize, tag` while our AST stores `tag, size` — the loader handles this reordering.

Exact match on semantics.

Status. **OK**

Risk. **Low**

8.4 MAKEBLOCK1

PDF Spec. MAKEBLOCK1 *t*: 1-field block.

```

690   Instruct(MAKEBLOCK1): {
691       tag_t tag = *pc++;
692       value block;
693       Alloc_small(block, 1, tag);
694       Field(block, 0) = accu;
695       accu = block;
696       Next;

```

```

442 | MAKEBLOCK1 t ⇒
443   let '(s', ptr)    heap_alloc s t [s.(accu)] in
444   Step (st s' pc' ptr s.(stack) s.(env) s.(extra_args) s.(global)
        s.(trap_sp))

```

Analysis. Single-field block with accu. Exact match.

Status. OK

Risk. Low

8.5 MAKEBLOCK2

PDF Spec. MAKEBLOCK2 *t*: 2-field block.

```

698 Instruct(MAKEBLOCK2): {
699   tag_t tag = *pc++;
700   value block;
701   Alloc_small(block, 2, tag);
702   Field(block, 0) = accu;
703   Field(block, 1) = sp[0];
704   sp += 1;
705   accu = block;
706   Next;

```

```

446 | MAKEBLOCK2 t ⇒
447   match s.(stack) with
448   | v1 :: rest ⇒
449     let '(s', ptr)    heap_alloc s t [s.(accu); v1] in
450     Step (st s' pc' ptr rest s.(env) s.(extra_args) s.(global)
          s.(trap_sp))
451 | _ ⇒ Error "MAKEBLOCK2: stack underflow"

```

Analysis. [accu, sp[0]]. Exact match.

Status. OK

Risk. Low

8.6 MAKEBLOCK3

PDF Spec. MAKEBLOCK3 *t*: 3-field block.

```

708   Instruct(MAKEBLOCK3): {
709       tag_t tag = *pc++;
710       value block;
711       Alloc_small(block, 3, tag);
712       Field(block, 0) = accu;
713       Field(block, 1) = sp[0];
714       Field(block, 2) = sp[1];
715       sp += 2;
716       accu = block;
717       Next;

```

```

454 | MAKEBLOCK3 t ⇒
455   match s.(stack) with
456 | v1 :: v2 :: rest ⇒
457   let '(s', ptr)    heap_alloc s t [s.(accu); v1; v2] in
458   Step (st s' pc' ptr rest s.(env) s.(extra_args) s.(global)
         s.(trap_sp))
459 | _ ⇒ Error "MAKEBLOCK3: stack underflow"
460
461 (* MAKEFLOATBLOCK n: accu=field0, pop (n-1) from stack.
462    Use tag 254 (double array) to hold float fields. *)

```

Analysis. [accu, sp[0], sp[1]]. Exact match.

Status. OK

Risk. Low

8.7 MAKEFLOATBLOCK

PDF Spec. MAKEFLOATBLOCK n : Allocates a block of n boxed floats with tag Double_array_tag (254).

```

719   Instruct(MAKEFLOATBLOCK): {
720       msize_t size = *pc++;
721       msize_t i;
722       value block;
723       if (size <= Max_young_wosize / Double_wosize) {
724         Alloc_small(block, size * Double_wosize, Double_array_tag);
725       } else {
726         block = caml_alloc_shr(size * Double_wosize,
                                Double_array_tag);
727       }
728       Store_double_flat_field(block, 0, Double_val(accu));
729       for (i = 1; i < size; i++){
730         Store_double_flat_field(block, i, Double_val(*sp));
731         ++ sp;
732       }
733       accu = block;
734       Next;

```

```

464 | MAKEFLOATBLOCK n ⇒
465   let fields      s.(accu) :: firstn (Nat.sub n 1) s.(stack) in
466   let new_stack    skipn (Nat.sub n 1) s.(stack) in
467   let '(s', ptr)   heap_alloc s 254 fields in
468   Step (st s' pc' ptr new_stack s.(env) s.(extra_args) s.(global)
        s.(trap_sp))

```

Analysis. C allocates with `size * Double_wosize` words and uses `Store_double_flat_field` for flat (unboxed) float storage. Rocq allocates with tag 254 and stores values directly (no float unboxing). This is a simplification: our model does not distinguish boxed from unboxed float representation. Since OCaml bytecode programs that use floats will have their values go through C primitives for arithmetic, and we model C calls via `CCall_request`, this simplification is acceptable for integer-only programs but would need revision for float-heavy code.

Status. **NEEDS TEST:** Float representation differs from C; needs testing with float-using programs.

Risk. **Medium**—Simplified float model.

9 Field Access

9.1 GETFIELD

PDF Spec. `GETFIELD n`: `accu = Field(accu, n)`.

```

747   Instruct(GETFIELD):
748     accu = Field(accu, *pc); pc++; Next;

```

```

470 | GETFIELD n ⇒
471   match field_or_heap s s.(accu) n with
472   | Some v ⇒ Step (st s pc' v s.(stack) s.(env) s.(extra_args)
                    s.(global) s.(trap_sp))
473   | None  ⇒ Error "GETFIELD: access failed"
474
475   (* GETFLOATFIELD n: accu is a float array (tag 254), get field n.
        *)

```

Analysis. `field_or_heap s accu n`. Handles both inline blocks and heap pointers. Exact match.

Status. **OK**

Risk. **Low**

9.2 GETFLOATFIELD

PDF Spec. `GETFLOATFIELD n`: Reads the n -th double from a float array block and boxes it.

```

749   Instruct(GETFLOATFIELD): {
750       double d = Double_flat_field(accu, *pc++);
751       Alloc_small(accu, Double_wosize, Double_tag);
752       Store_double_val(accu, d);
753       Next;

```

```

477 | GETFLOATFIELD n ⇒
478   match field_or_heap s s.(accu) n with
479 | Some v ⇒ Step (st s pc' v s.(stack) s.(env) s.(extra_args)
480   s.(global) s.(trap_sp))
480 | None ⇒ Error "GETFLOATFIELD: access failed"

```

Analysis. C: reads flat double, allocates new box (Double_tag). Rocq: just reads the field (no boxing/unboxing distinction). Same simplification as MAKEFLOATBLOCK.

Status. **NEEDS TEST:** Float boxing semantics simplified.

Risk. **Medium**— Float model simplification.

9.3 SETFIELD

PDF Spec. SETFIELD n : Field(accu, n) = sp[0]; sp++; accu = (). Mutates field n of the block in accu with the value from stack top.

```

772   Instruct(SETFIELD):
773       caml_modify(&Field(accu, *pc), *sp++);
774       accu = Val_unit;
775       pc++;
776       Next;

```

```

483 | SETFIELD n ⇒
484   match s.(stack) with
485 | newval :: rest ⇒
486   match s.(accu) with
487 | Val_ptr addr ⇒
488   match heap_lookup s.(hp) addr with
489 | Some (_, fields) ⇒
490   match set_nth fields n newval with
491 | Some new_fields ⇒
492   let new_hp      heap_update s.(hp) addr new_fields in
493   Step (mk_state pc' val_unit rest s.(env) s.(extra_args)
494     s.(global) s.(trap_sp) new_hp s.(next_addr))
494 | None ⇒ Error "SETFIELD: index out of bounds"
495 | None ⇒ Error "SETFIELD: dangling pointer"
496 | _ ⇒ Error "SETFIELD: not a mutable block"
497 | _ ⇒ Error "SETFIELD: stack underflow"
498
499 (* SETFLOATFIELD n: stack top is the float array (tag 254), accu
   is the new float value. *)

```

Analysis. C: `caml_modify(&Field(accu, *pc), *sp++); accu = Val_unit`. Rocq: pops new value from stack, looks up heap block, uses `set_nth` on fields, then `heap_update`.

Important: In the C code, `accu` is the block and `*sp` is the new value. In the Rocq code (line 484), the new value comes from the stack (`newval :: rest`), and `accu (Val_ptr addr)` is the block. This matches the C semantics correctly.

Status. **OK**

Risk. **Low**

9.4 SETFLOATFIELD

PDF Spec. `SETFLOATFIELD n`: Stores a float into field `n` of a float array. `Store_double_flat_field(accu, n, Double_val(*sp)); sp++; accu = ()`.

```

777   Instruct(SETFLOATFIELD):
778       Store_double_flat_field(accu, *pc, Double_val(*sp));
779       accu = Val_unit;
780       sp++;
781       pc++;
782       Next;

```

```

504   | SETFLOATFIELD n =>
505       match s.(stack) with
506       | arr_val :: rest =>
507           match arr_val with
508           | Val_ptr addr =>
509               match heap_lookup s.(hp) addr with
510               | Some (_, fields) =>
511                   match set_nth fields n s.(accu) with
512                   | Some new_fields =>
513                       let new_hp      heap_update s.(hp) addr new_fields in
514                       Step (mk_state pc' val_unit rest s.(env) s.(extra_args)
515                             s.(global) s.(trap_sp) new_hp s.(next_addr))
515               | None => Error "SETFLOATFIELD: index out of bounds"
516           | None => Error "SETFLOATFIELD: dangling pointer"
517       | _ => Error "SETFLOATFIELD: not a heap float array"
518       | _ => Error "SETFLOATFIELD: stack underflow"

```

Analysis. **CRITICAL: Potential operand inversion.**

In `interp.c` (line 778): `Store_double_flat_field(accu, *pc, Double_val(*sp))`. Here `accu` is the float array and `*sp` is the new float value.

In our Rocq code (line 504–521), the code does:

```
match s.(stack) with | arr_val :: rest => ...
```

So `arr_val` (from the stack) is treated as the float array, and `s.(accu)` is the new value. This is the **opposite** convention from `interp.c`!

However, looking more carefully at the comment on line 503: “*SETFLOATFIELD n: stack top is the float array (tag 254), accu is the new float value.*” This comment explicitly states the inverted convention.

Verdict. The Rocq code has `accu = new value` and `sp[0] = array`. The C code has `accu = array` and `sp[0] = new value`. This is a **bug** if both conventions are used with the same bytecode.

BUT: Looking at the regular `SETFIELD` (line 483), it correctly uses `accu = block` and `sp = new value`. `SETFLOATFIELD` should follow the same convention. The inversion in our Rocq code is indeed a bug.

Status. **BUG:** Operand convention inverted: `accu` should be the float array, `sp[0]` should be the new value, but our code swaps them.

Risk. **High:** Affects all programs using mutable float arrays. May be masked by PBT if float programs are rare in the test suite.

10 Array and String Operations

10.1 VECTLENGTH

PDF Spec. `VECTLENGTH`: `accu = Wosize_val(accu)`. For double arrays, divides by `Double_wosize`.

```
786   Instruct(VECTLENGTH): {
787       /* Todo: when FLAT_FLOAT_ARRAY is false, this instruction
788          should
789          be split into VECTLENGTH and FLOATVECTLENGTH because we know
790          statically which one it is. */
791       msize_t size = Wosize_val(accu);
792       if (Tag_val(accu) == Double_array_tag) size = size /
793           Double_wosize;
794       accu = Val_long(size);
795       Next;
```

```
524 | VECTLENGTH ⇒
525 | match size_or_heap s s.(accu) with
526 | Some n ⇒ Step (st s pc' (Val_int (Z.of_nat n)) s.(stack)
527 | None ⇒ Error "VECTLENGTH: not a block"
528 | None ⇒ Error "VECTLENGTH: not a block"
```

Analysis. C: checks tag for `Double_array_tag` and divides `size`. Rocq: uses `size_or_heap` which returns the number of fields directly. Since our float arrays store one value per field (no flat representation), the count is already correct without division.

Status. **OK**

Risk. **Low**

10.2 GETVECTITEM

PDF Spec. `GETVECTITEM`: `accu = Field(accu, Long_val(sp[0]))`; `sp++`.


```

795   Instruct(GETVECTITEM):
796     accu = Field(accu, Long_val(sp[0]));
797     sp += 1;
798     Next;

```

```

530 | GETVECTITEM ⇒
531   match s.(stack) with
532   | Val_int idx :: rest ⇒
533     match field_or_heap s s.(accu) (Z.to_nat idx) with
534     | Some v ⇒ Step (st s pc' v rest s.(env) s.(extra_args)
535                     s.(global) s.(trap_sp))
535     | None ⇒ Error "GETVECTITEM: index out of bounds"
536   | _ ⇒ Error "GETVECTITEM: bad index or stack underflow"

```

Analysis. Index from stack top, field access on accu. Exact match.

Status. OK

Risk. Low

10.3 SETVECTITEM

PDF Spec. SETVECTITEM: Field(accu, Long_val(sp[0])) = sp[1]; sp+=2; accu = ().

```

799   Instruct(SETVECTITEM):
800     caml_modify(&Field(accu, Long_val(sp[0])), sp[1]);
801     accu = Val_unit;
802     sp += 2;
803     Next;

```

```

540 | SETVECTITEM ⇒
541   match s.(stack) with
542   | Val_int idx :: newval :: rest ⇒
543     match s.(accu) with
544     | Val_ptr addr ⇒
545       match heap_lookup s.(hp) addr with
546       | Some (_, fields) ⇒
547         match set_nth fields (Z.to_nat idx) newval with
548         | Some new_fields ⇒
549           let new_hp = heap_update s.(hp) addr new_fields in
550           Step (mk_state pc' val_unit rest s.(env) s.(extra_args)
551                 s.(global) s.(trap_sp) new_hp s.(next_addr))
551         | None ⇒ Error "SETVECTITEM: index out of bounds"
552       | None ⇒ Error "SETVECTITEM: dangling pointer"
553     | _ ⇒ Error "SETVECTITEM: not a heap block"
554   | _ ⇒ Error "SETVECTITEM: stack underflow"

```

Analysis. C: sp[0] = index, sp[1] = new value. Rocq: Val_int idx :: newval :: rest. Same order. Mutates via heap_update. Exact match.

Status. **OK**

Risk. **Low**

10.4 GETSTRINGCHAR

PDF Spec. GETSTRINGCHAR/GETBYTESCHAR: `accu = Val_int(Byte_u(accu, Long_val(sp[0]))); sp++`.

```
806 | Instruct(GETSTRINGCHAR):
```

```
560 | GETBYTESCHAR | GETSTRINGCHAR =>
561 | match s.(stack) with
562 | Val_int idx :: rest =>
563 | match field_or_heap s s.(accu) (Z.to_nat idx) with
564 | Some (Val_int c) => Step (st s pc' (Val_int c) rest s.(env)
    s.(extra_args) s.(global) s.(trap_sp))
565 | _ => Error "GETSTRINGCHAR: index out of bounds or not a char"
566 | _ => Error "GETSTRINGCHAR: stack underflow"
```

Analysis. Our model represents strings as blocks of integer character values (not raw bytes). The code extracts a `Val_int c` at the given index. This is a simplification—real OCaml strings are packed byte arrays. The simplification is valid as long as string operations go through C primitives and our loader correctly unpacks strings.

Both GETSTRINGCHAR and GETBYTESCHAR share the same case in our code, matching the C implementation.

Status. **OK**

Risk. **Low**

10.5 GETBYTESCHAR

PDF Spec. Same as GETSTRINGCHAR.

Analysis. Combined with GETSTRINGCHAR in both C and Rocq.

Status. **OK**

Risk. **Low**

10.6 SETBYTESCHAR

PDF Spec. SETBYTESCHAR: `Byte_u(accu, Long_val(sp[0])) = Int_val(sp[1]); sp+=2; accu = ()`.

```

811   Instruct (SETBYTESCHAR):
812     Byte_u(accum, Long_val(sp[0])) = Int_val(sp[1]);
813     sp += 2;
814     accum = Val_unit;
815     Next;

```

```

570 | SETBYTESCHAR ⇒
571   match s.(stack) with
572   | Val_int idx :: Val_int newchar :: rest ⇒
573     match s.(accum) with
574     | Val_ptr addr ⇒
575       match heap_lookup s.(hp) addr with
576       | Some (_, fields) ⇒
577         match set_nth fields (Z.to_nat idx) (Val_int newchar) with
578         | Some new_fields ⇒
579           let new_hp      heap_update s.(hp) addr new_fields in
580           Step (mk_state pc' val_unit rest s.(env) s.(extra_args)
                 s.(global) s.(trap_sp) new_hp s.(next_addr))
581         | None ⇒ Error "SETBYTESCHAR: index out of bounds"
582       | None ⇒ Error "SETBYTESCHAR: dangling pointer"
583     | _ ⇒ Error "SETBYTESCHAR: not a heap bytes"
584   | _ ⇒ Error "SETBYTESCHAR: stack underflow"

```

Analysis. Heap mutation for byte arrays. Our model uses `set_nth` on the field list. Functionally equivalent given our string-as-block-of-ints representation.

Status. OK

Risk. Low

11 Control Flow

11.1 BRANCH

PDF Spec. `BRANCH ofs: pc += ofs.`

```

819   Instruct (BRANCH):
820     pc += *pc;
821     Next;

```

```

590 | BRANCH target ⇒
591   Step (st s target s.(accum) s.(stack) s.(env) s.(extra_args)
         s.(global) s.(trap_sp))

```

Analysis. C: `pc += *pc` (relative offset). Rocq: `pc = target` (absolute, pre-resolved by loader). Equivalent after loader processing.

Status. **OK**

Risk. **Low**

11.2 BRANCHIF

PDF Spec. *BRANCHIF ofs*: Branch if `accu != Val_false` (i.e., $\neq 0$).

```
822 | Instruct(BRANCHIF):  
823 |     if (accu != Val_false) pc += *pc; else pc++;  
824 |     Next;
```

```
593 | | BRANCHIF target ⇒  
594 | | match s.(accu) with  
595 | | | Val_int 0 ⇒ Step (st s pc' s.(accu) s.(stack) s.(env)  
596 | | |   s.(extra_args) s.(global) s.(trap_sp))  
596 | | | _ ⇒ Step (st s target s.(accu) s.(stack) s.(env)  
596 | | |   s.(extra_args) s.(global) s.(trap_sp))
```

Analysis. C: `accu != Val_false`. `Val_false = Val_int(0)`. Rocq: matches `Val_int 0` for the no-branch case, wildcard for branch. This is correct: any non-`Val_int 0` value (including blocks) triggers the branch, matching the C `Is_block` behavior.

Status. **OK**

Risk. **Low**

11.3 BRANCHIFNOT

PDF Spec. *BRANCHIFNOT ofs*: Branch if `accu == Val_false`.

```
825 | Instruct(BRANCHIFNOT):  
826 |     if (accu == Val_false) pc += *pc; else pc++;  
827 |     Next;
```

```
599 | | BRANCHIFNOT target ⇒  
600 | | match s.(accu) with  
601 | | | Val_int 0 ⇒ Step (st s target s.(accu) s.(stack) s.(env)  
602 | | |   s.(extra_args) s.(global) s.(trap_sp))  
602 | | | _ ⇒ Step (st s pc' s.(accu) s.(stack) s.(env) s.(extra_args)  
602 | | |   s.(global) s.(trap_sp))
```

Analysis. Dual of `BRANCHIF`. Exact match.

Status. **OK**

Risk. **Low**

11.4 SWITCH

PDF Spec. SWITCH: Dispatches on `accu`. The operand word encodes `nc` (number of constant cases, low 16 bits) and `nb` (number of block cases, high 16 bits). If `accu` is an integer, jump to `pc[Long_val(accu)]`. If a block, jump to `pc[nc + Tag_val(accu)]`.

```
828 Instruct(SWITCH): {
829     uint32_t sizes = *pc++;
830     if (Is_block(accu)) {
831         intnat index = Tag_val(accu);
832         CAMLassert ((uintnat) index < (sizes >> 16));
833         pc += pc[(sizes & 0xFFFF) + index];
834     } else {
835         intnat index = Long_val(accu);
836         CAMLassert ((uintnat) index < (sizes & 0xFFFF)) ;
837         pc += pc[index];
838     }
839     Next;
```

```
605 | SWITCH_nc_nb const_targets block_targets =>
606 match s.(accu) with
607 | Val_int n =>
608     match nth_error const_targets (Z.to_nat n) with
609     | Some target => Step (st s target s.(accu) s.(stack) s.(env)
610                          s.(extra_args) s.(global) s.(trap_sp))
611     | None => Error "SWITCH: constant index out of range"
612 | Val_block t _ =>
613     match nth_error block_targets t with
614     | Some target => Step (st s target s.(accu) s.(stack) s.(env)
615                          s.(extra_args) s.(global) s.(trap_sp))
616     | None => Error "SWITCH: block tag out of range"
617 | Val_ptr addr | Val_closure addr _ =>
618     match tag_or_heap s s.(accu) with
619     | Some t =>
620         match nth_error block_targets t with
621         | Some target => Step (st s target s.(accu) s.(stack)
622                              s.(env) s.(extra_args) s.(global) s.(trap_sp))
623         | None => Error "SWITCH: block tag out of range"
624         | None => Error "SWITCH: dangling pointer"
```

Analysis. C: `sizes = *pc++`; if `(Is_block(accu)) index = Tag_val(accu)`, `pc += pc[(sizes & 0xFFFF) + index]`; else `index = Long_val(accu)`, `pc += pc[index]`.

Rocq: The instruction is pre-decoded with separate `const_targets` and `block_targets` lists. For integers, looks up `nth_error const_targets (Z.to_nat n)`. For blocks/pointers, uses `tag_or_heap` to get the tag, then `nth_error block_targets t`.

The key difference is that the loader splits the single offset table into two separate lists, while `interp.c` uses a combined table with `nc` as the split point. Functionally equivalent assuming the loader correctly splits at `nc`.

Status. **OK**

Risk. **Low**— Depends on correct loader splitting.

11.5 BOOLNOT

PDF Spec. BOOLNOT: `accu = Val_not(accu)`.

```
841   Instruct(BOOLNOT):
842       accu = Val_not(accu);
843       Next;
```

```
628   | BOOLNOT =>
629       match s.(accu) with
630       | Val_int 0 => Step (st s pc' val_true s.(stack) s.(env)
631                           s.(extra_args) s.(global) s.(trap_sp))
631       | _ => Step (st s pc' val_false s.(stack) s.(env) s.(extra_args)
632                   s.(global) s.(trap_sp))
632
633   (* PUSHTRAP: push trap frame [handler_pc, prev_trap_sp, env,
634     extra_args] onto
634     the stack, then set trap_sp = length(new_stack).
635     Layout mirrors interp.c: sp[0]=handler_pc, sp[1]=trap_link,
636     sp[2]=env, sp[3]=extra_args. *)
```

Analysis. C: `Val_not(accu)` flips the tag bit. Rocq: pattern matches on `Val_int 0` to produce `val_true/val_false`. Exact match for boolean values.

Status. **OK**

Risk. **Low**

12 Exception Handling

12.1 PUSHTRAP

PDF Spec. PUSHTRAP *ofs*: Push trap frame [`handler_pc`, `trap_link`, `env`, `extra_args`] and set `trapsp = sp`.

```
847   Instruct(PUSHTRAP):
848       sp -= 4;
849       Trap_pc(sp) = pc + *pc;
850       Trap_link_offset(sp) = Val_long(Caml_state->trapsp - sp);
851       sp[2] = env;
852       sp[3] = Val_long(extra_args);
853       Caml_state->trapsp = sp;
854       pc++;
855       Next;
```

```

637 | PUSHTRAP handler_pc =>
638   let prev_tsp      Val_int (Z.of_nat s.(trap_sp)) in
639   let new_stack      Val_int handler_pc :: prev_tsp :: s.(env) ::
        Val_int (Z.of_nat s.(extra_args)) :: s.(stack) in
640   let new_tsp      length new_stack in
641   Step (st s pc' s.(accu) new_stack s.(env) s.(extra_args)
        s.(global) new_tsp)
642
643   (* POPTRAP: restore trap_sp from the trap link (sp[1]) and pop 4
        values.
644     For well-formed code the trap frame is always at the current
        stack top. *)

```

Analysis. C trap frame: $sp[0] = pc+*pc$ (handler), $sp[1] = \text{Val_long}(\text{trap_sp} - sp)$ (link as offset), $sp[2] = env$, $sp[3] = \text{Val_long}(\text{extra_args})$. Then $\text{trap_sp} = sp$.

Rocq trap frame: $\text{Val_int handler_pc} :: \text{Val_int}(\text{trap_sp}) :: env :: \text{Val_int}(\text{extra_args}) :: stack$. Then $\text{trap_sp} = \text{length}(\text{new_stack})$.

Trap link encoding:

- C stores $\text{trap_sp} - sp$ — the *offset* from current sp to previous trap frame.
- Rocq stores the *absolute* previous trap_sp value (stack depth).

These are equivalent representations. The C offset plus the current sp gives the previous trap_sp . Our absolute value directly records it. On POPTRAP, the C code recovers $\text{trap_sp} = sp + \text{Long_val}(\text{Trap_link_offset}(sp))$, while we directly restore the saved trap_sp value.

Status. OK

Risk. Medium— The trap link representation differs; any bug in the offset computation would be subtle.

12.2 POPTRAP

PDF Spec. POPTRAP: Restore trap_sp from the trap link and pop the 4-word trap frame.

```

857   Instruct(POPTRAP):
858     if (caml_something_to_do) {
859       /* We must check here so that if a signal is pending and its
860          handler triggers an exception, the exception is trapped
861          by the current try...with, not the enclosing one. */
862       pc--; /* restart the POPTRAP after processing the signal */
863       goto process_actions;
864     }
865     Caml_state->trap_sp = sp + Long_val(Trap_link_offset(sp));
866     sp += 4;
867     Next;

```

```

645 | POPTRAP ⇒
646   match s.(stack) with
647   | _ :: Val_int prev_tsp :: _ :: _ :: rest ⇒
648     Step (st s pc' s.(accu) rest s.(env) s.(extra_args) s.(global)
           (Z.to_nat prev_tsp))
649   | _ ⇒ Error "POPTRAP: malformed trap frame"
650
651   (* RAISE/RERAISE/RAISE_NOTRACE: restore sp to the trap frame,
     extract
652     handler_pc / prev_trap_sp / env / extra_args from the frame,
     and jump.
653     In interp.c: sp = trapsp; pc = Trap_pc(sp); trapsp = sp+link;
     env=sp[2]; ea=sp[3]; sp+=4. *)

```

Analysis. C: `trapsp = sp + Long_val(Trap_link_offset(sp)); sp += 4`. Also checks `caml_something_` (signal handling) which we skip.

Rocq: Pattern-matches the trap frame, extracts `prev_tsp` from position 1, pops 4 values. Sets `trap_sp = Z.to_nat prev_tsp`.

Exact match modulo signal handling (which is a no-op in our model).

Status. OK

Risk. Low

12.3 RAISE

PDF Spec. RAISE: Raise the exception in `accu`. Restores `sp` to `trapsp`, extracts handler `pc`, restores previous trap link, `env`, and `extra_args`.

```

881 Instruct(RAISE):
882   Check_trap_barrier;
883   if (Caml_state->backtrace_active) {
884     *--sp = (value)(pc - 1);
885     caml_stash_backtrace(accu, sp, 0);
886   }
887   raise_notrace:
888   if ((char *) Caml_state->trapsp
889       >= (char *) Caml_state->stack_high - initial_sp_offset) {
890     Caml_state->external_raise = initial_external_raise;
891     Caml_state->extern_sp = (value *) ((char *)
892                                       Caml_state->stack_high
893                                       - initial_sp_offset);
894     caml_callback_depth--;
895     return Make_exception_result(accu);
896   }
897   sp = Caml_state->trapsp;
898   pc = Trap_pc(sp);
899   Caml_state->trapsp = sp + Long_val(Trap_link_offset(sp));
900   env = sp[2];
901   extra_args = Long_val(sp[3]);
902   sp += 4;
903   Next;

```



```

655 | RAISE | RERAISE | RAISE_NOTRACE ⇒
656 | do_raise s.(accu) s

```

Analysis. All three raise variants (RAISE, RERAISE, RAISE_NOTRACE) call `do_raise` in our code. The C versions differ only in backtrace handling (which we don’t model).

`do_raise`: If `trap_sp = 0`, returns `Error "unhandled exception"`. Otherwise computes `k = length(stack) - trap_sp`, skips `k` elements to reach the trap frame, extracts `[handler_pc, prev_tsp, saved_env, saved_ea]`, and jumps.

The C code: `sp = trapsp; pc = Trap_pc(sp); trapsp = sp + Long_val(Trap_link_offset(sp)); env = sp[2]; extra_args = Long_val(sp[3]); sp += 4.`

Equivalent: our skip `n` stack positions at the trap frame top, then pattern-matches the 4-word frame.

Edge case: C checks if `trapsp` is beyond the initial stack (unhandled exception in callback) via pointer comparison. Our check `trap_sp = 0` is the equivalent sentinel for “no handler”.

Status. **OK**

Risk. **Medium**— Trap frame navigation is the most subtle part of the interpreter.

12.4 RERAISE

PDF Spec. Same as RAISE but with backtrace re-raise semantics.

Analysis. Same as RAISE in our model (no backtrace).

Status. **OK**

Risk. **Low**

12.5 RAISE_NOTRACE

PDF Spec. Same as RAISE but without backtrace recording.

Analysis. Same as RAISE in our model.

Status. **OK**

Risk. **Low**

13 Signal Handling and C Calls

13.1 CHECK_SIGNALS

PDF Spec. CHECK_SIGNALS: Process pending signals.

```

916 | Instruct(CHECK_SIGNALS):      /* accu not preserved */
917 |     if (caml_something_to_do) goto process_actions;
918 |     Next;

```

```

658 | CHECK_SIGNALS ⇒
659   Step (st s pc' s.(accu) s.(stack) s.(env) s.(extra_args)
        s.(global) s.(trap_sp))
660
661   (* C_CALL: suspend and request a C primitive call. *)

```

Analysis. Modeled as no-op (just advance PC). In the C interpreter, this checks `caml_something_to_do` and processes pending actions. Safe to skip since we don't model OS signals.

Status. **OK**

Risk. **Low**

13.2 C_CALL

PDF Spec. `C_CALL n p`: Call C primitive `p` with `n` arguments. The C code has specialized forms `C_CALL1` through `C_CALL5` and a generic `C_CALLN`.

```

928   Instruct(C_CALL1):
929     Setup_for_c_call;
930     accu = Primitive1(*pc)(accu);
931     Restore_after_c_call;
932     pc++;
933     Next;

```

```

662 | C_CALL nargs prim_idx ⇒
663   let args      s.(accu) :: firstn (Nat.sub nargs 1) s.(stack) in
664   let new_stack  skipn (Nat.sub nargs 1) s.(stack) in
665   let cont      st s pc' val_unit new_stack s.(env) s.(extra_args)
        s.(global) s.(trap_sp) in
666   CCall_request prim_idx args cont

```

Analysis. C: sets up frame, calls primitive, restores frame. Rocq: returns `CCall_request prim_idx args cont`, suspending execution. The caller must supply the primitive's return value to resume.

This is a fundamental architectural difference — we don't execute C primitives but instead produce an interaction-tree-style request. The PBT harness fulfills these by calling the actual C primitives.

Status. **OK**

Risk. **Low**— The interaction tree model is well-understood.

14 Integer Constants

14.1 CONSTINT

PDF Spec. `CONSTINT n`: `accu = Val_int(n)`. Specialized: `CONST0`–`CONST3`.

```

996   Instruct(CONSTINT):
997       accu = Val_int(*pc);
998       pc++;
999       Next;

```

```

668 | CONSTANT n ⇒
669   Step (st s pc' (Val_int n) s.(stack) s.(env) s.(extra_args)
        s.(global) s.(trap_sp))

```

Analysis. Direct assignment. Exact match.

Status. OK

Risk. Low

14.2 PUSHCONSTINT

PDF Spec. PUSHCONSTINT n : Push accu, then CONSTINT.

```

993   Instruct(PUSHCONSTINT):
994       *--sp = accu;
995       /* Fallthrough */

```

```

671 | PUSHCONSTINT n ⇒
672   let new_stack = s.(accu) :: s.(stack) in
673   Step (st s pc' (Val_int n) new_stack s.(env) s.(extra_args)
        s.(global) s.(trap_sp))

```

Analysis. Push + constint. Exact match.

Status. OK

Risk. Low

15 Integer Arithmetic

15.1 NEGINT

PDF Spec. NEGINT: $\text{accu} = -\text{accu}$ (integer negation).

```

1003   Instruct(NEGINT):
1004       accu = (value)(2 - (intnat)accu); Next;

```

```

675 | NEGINT ⇒
676   match s.(accu) with
677   | Val_int n ⇒ Step (st s pc' (Val_int (- n)) s.(stack) s.(env)
678                     s.(extra_args) s.(global) s.(trap_sp))
        | _ ⇒ Error "NEGINT: not an integer"

```

Analysis. C: $(\text{value}) (2 - (\text{intnat})\text{accu})$ — this works because for tagged integers, $\text{Val_int}(n) = 2n+1$, so $2 - (2n+1) = -(2n-1) = 2(-n)+1 = \text{Val_int}(-n)$. Rocq: $\text{Val_int}(-n)$. Exact match.

Status. OK

Risk. Low

15.2 ADDINT

PDF Spec. ADDINT: $\text{accu} = \text{accu} + \text{sp}[0]; \text{sp}++$.

```

1005 Instruct(ADDINT):
1006   accu = (value)((intnat) accu + (intnat) *sp++ - 1); Next;

```

```

681 | ADDINT ⇒
682   match s.(accu), s.(stack) with
683   | Val_int a, Val_int b :: rest ⇒ Step (st s pc' (Val_int (a +
684                                     b)) rest s.(env) s.(extra_args) s.(global) s.(trap_sp))
        | _, _ ⇒ Error "ADDINT: type error or stack underflow"

```

Analysis. C: $(\text{intnat})\text{accu} + (\text{intnat})*\text{sp}++ - 1$ (tag bit arithmetic). Rocq: $\text{Val_int}(a + b)$. Exact match at the semantic level.

Status. OK

Risk. Low

15.3 SUBINT

PDF Spec. SUBINT: $\text{accu} = \text{accu} - \text{sp}[0]; \text{sp}++$.

```

1007 Instruct(SUBINT):
1008   accu = (value)((intnat) accu - (intnat) *sp++ + 1); Next;

```

```

687 | SUBINT ⇒
688   match s.(accu), s.(stack) with
689   | Val_int a, Val_int b :: rest ⇒ Step (st s pc' (Val_int (a -
690                                     b)) rest s.(env) s.(extra_args) s.(global) s.(trap_sp))
        | _, _ ⇒ Error "SUBINT: type error or stack underflow"

```

Analysis. C: (intnat)accu - (intnat)*sp++ + 1. Rocq: a - b. Exact match.

Status. OK

Risk. Low

15.4 MULINT

PDF Spec. MULINT: accu = accu * sp[0]; sp++.

```
1009   Instruct(MULINT):  
1010     accu = Val_long(Long_val(accu) * Long_val(*sp++)); Next;
```

```
693   | MULINT ⇒  
694     match s.(accu), s.(stack) with  
695     | Val_int a, Val_int b :: rest ⇒ Step (st s pc' (Val_int (a *  
696       b)) rest s.(env) s.(extra_args) s.(global) s.(trap_sp))  
696     | _, _ ⇒ Error "MULINT: type error or stack underflow"
```

Analysis. C: Val_long(Long_val(accu) * Long_val(*sp++)). Rocq: a * b. Exact match.

Status. OK

Risk. Low

15.5 DIVINT

PDF Spec. DIVINT: accu = accu / sp[0]; sp++. Raises Division_by_zero if divisor is 0.

```
1012   Instruct(DIVINT): {  
1013     intnat divisor = Long_val(*sp++);  
1014     if (divisor == 0) { Setup_for_c_call;  
1015       caml_raise_zero_divide(); }  
1015     accu = Val_long(Long_val(accu) / divisor);  
1016     Next;
```

```
699   | DIVINT ⇒  
700     match s.(accu), s.(stack) with  
701     | Val_int a, Val_int b :: rest ⇒  
702       if Z.eqb b 0 then do_raise div_by_zero_exn s  
703       else Step (st s pc' (Val_int (Z.quot a b)) rest s.(env)  
704         s.(extra_args) s.(global) s.(trap_sp))  
704     | _, _ ⇒ Error "DIVINT: type error or stack underflow"
```

Analysis. C: checks `divisor == 0`, calls `caml_raise_zero_divide()`. Rocq: checks `Z.eqb b 0`, calls `do_raise div_by_zero_exn`.

Both raise an exception on zero. The Rocq exception is a manually constructed `Division_by_zero` block (tag 248, id -6), matching OCaml's predefined exception.

Division uses `Z.quot` (truncation toward zero), matching C's `/` operator for signed integers.

Status. OK

Risk. Low

15.6 MODINT

PDF Spec. MODINT: `accu = accu % sp[0]; sp++`. Raises on zero.

```
1018   Instruct(MODINT): {
1019       intnat divisor = Long_val(*sp++);
1020       if (divisor == 0) { Setup_for_c_call;
1021           caml_raise_zero_divide(); }
1021       accu = Val_long(Long_val(accu) % divisor);
1022       Next;
```

```
707 | MODINT ⇒
708   match s.(accu), s.(stack) with
709   | Val_int a, Val_int b :: rest ⇒
710     if Z.eqb b 0 then do_raise div_by_zero_exn s
711     else Step (st s pc' (Val_int (Z.rem a b)) rest s.(env)
712               s.(extra_args) s.(global) s.(trap_sp))
712 | _, _ ⇒ Error "MODINT: type error or stack underflow"
```

Analysis. Same pattern as DIVINT. Uses `Z.rem` which matches C's `%` (remainder with truncation toward zero).

Status. OK

Risk. Low

15.7 ANDINT

PDF Spec. ANDINT: Bitwise AND.

```
1024   Instruct(ANDINT):
1025       accu = (value)((intnat) accu & (intnat) *sp++); Next;
```

```

715 | ANDINT ⇒
716 | match s.(accu), s.(stack) with
717 | Val_int a, Val_int b :: rest ⇒ Step (st s pc' (Val_int
    (Z.land a b)) rest s.(env) s.(extra_args) s.(global)
    s.(trap_sp))
718 | _, _ ⇒ Error "ANDINT: type error or stack underflow"

```

Analysis. C: (intnat)accu & (intnat)*sp++. Works directly on tagged values because AND preserves the tag bit. Rocq: Z.land a b. Exact match at the value level.

Status. OK

Risk. Low

15.8 ORINT

PDF Spec. ORINT: Bitwise OR.

```

1026 | Instruct(ORINT):
1027 |   accu = (value)((intnat) accu | (intnat) *sp++); Next;

```

```

721 | ORINT ⇒
722 | match s.(accu), s.(stack) with
723 | Val_int a, Val_int b :: rest ⇒ Step (st s pc' (Val_int (Z.lor
    a b)) rest s.(env) s.(extra_args) s.(global) s.(trap_sp))
724 | _, _ ⇒ Error "ORINT: type error or stack underflow"

```

Analysis. C: (intnat)accu | (intnat)*sp++. Preserves tag bit. Rocq: Z.lor a b. Exact match.

Status. OK

Risk. Low

15.9 XORINT

PDF Spec. XORINT: Bitwise XOR.

```

1028 | Instruct(XORINT):
1029 |   accu = (value)((((intnat) accu ^ (intnat) *sp++) | 1)); Next;

```

```

727 | XORINT ⇒
728 | match s.(accu), s.(stack) with
729 | Val_int a, Val_int b :: rest ⇒ Step (st s pc' (Val_int
    (Z.lxor a b)) rest s.(env) s.(extra_args) s.(global)
    s.(trap_sp))
730 | _, _ ⇒ Error "XORINT: type error or stack underflow"

```

Analysis. C: $((\text{intnat})\text{accu} \wedge (\text{intnat})*\text{sp}++) \mid 1$ — the $\mid 1$ restores the tag bit. Rocq: $\text{Z.lxor } a \ b$. Exact match (tag bit is not part of our value representation).

Status. OK

Risk. Low

15.10 LSLINT

PDF Spec. LSLINT: Logical shift left.

```
1030   Instruct (LSLINT):
1031     accu = (value) (((intnat) accu - 1) << Long_val(*sp++)) + 1);
        Next;
```

```
733   | LSLINT =>
734     match s.(accu), s.(stack) with
735     | Val_int a, Val_int b :: rest => Step (st s pc' (Val_int
        (Z.shiftl a b)) rest s.(env) s.(extra_args) s.(global)
        s.(trap_sp))
736     | _, _ => Error "LSLINT: type error or stack underflow"
```

Analysis. C: $((\text{intnat})\text{accu} - 1) \ll \text{Long_val}(*\text{sp}++) + 1$. Rocq: $\text{Z.shiftl } a \ b$. The C version works on tagged values: subtract tag, shift, restore tag. Our version operates on untagged values directly. Exact match.

Status. OK

Risk. Low

15.11 LSRINT

PDF Spec. LSRINT: Logical (unsigned) shift right.

```
1032   Instruct (LSRINT):
1033     accu = (value) (((uintnat) accu) >> Long_val(*sp++)) | 1);
        Next;
```

```
739   | LSRINT =>
740     match s.(accu), s.(stack) with
741     | Val_int a, Val_int b :: rest => Step (st s pc' (Val_int (z_lsr
        a b)) rest s.(env) s.(extra_args) s.(global) s.(trap_sp))
742     | _, _ => Error "LSRINT: type error or stack underflow"
```

Analysis. C: $((\text{uintnat})\text{accu}) \gg \text{Long_val}(*\text{sp}++) \mid 1$ — unsigned shift, restore tag. Rocq: $\text{z_lsr } a \ b$ where z_lsr masks to 63 bits then does Z.shiftr . The z_unsigned function ensures the shift is logical (fills with zeros from the left). Exact match.

Status. **OK**

Risk. **Low**

15.12 ASRINT

PDF Spec. ASRINT: Arithmetic (signed) shift right.

```
1034 | Instruct(ASRINT):  
1035 |   accu = (value)((((intnat) accu) >> Long_val(*sp++)) | 1); Next;
```

```
745 | ASRINT ⇒  
746 |   match s.(accu), s.(stack) with  
747 |   | Val_int a, Val_int b :: rest ⇒ Step (st s pc' (Val_int  
       (Z.shiftr a b)) rest s.(env) s.(extra_args) s.(global)  
       s.(trap_sp))  
748 |   | _, _ ⇒ Error "ASRINT: type error or stack underflow"
```

Analysis. C: $((\text{intnat})\text{accu}) \gg \text{Long_val}(*\text{sp}++) \mid 1$ — signed shift (implementation-defined but effectively arithmetic on all OCaml platforms), restore tag. Rocq: $\text{Z.shiftr } a \ b$ — arithmetic right shift on Z. Exact match.

Status. **OK**

Risk. **Low**

16 Comparisons

16.1 EQ

PDF Spec. EQ: $\text{accu} = (\text{accu} == \text{sp}[0]); \text{sp}++$. Physical equality.

```
1041 | Integer_comparison(intnat, EQ, ==)
```

```
751 | EQ ⇒  
752 |   match s.(stack) with  
753 |   | b :: rest ⇒ Step (st s pc' (if value_eqb s.(accu) b then  
       val_true else val_false) rest s.(env) s.(extra_args)  
       s.(global) s.(trap_sp))  
754 |   | _ ⇒ Error "EQ: stack underflow"
```

Analysis. C: physical equality (pointer comparison). Rocq: `value_eqb` `accu sp[0]` which does structural comparison. For integers, these are identical. For heap pointers, structural equality is stricter than physical equality (two equal blocks at different addresses compare equal structurally but not physically).

This is a known simplification: our structural equality is a conservative approximation that may return `true` in cases where physical equality returns `false`. For well-typed OCaml programs, this difference is only observable with mutable values (which should use `==` sparingly).

Status. **NEEDS TEST:** Structural vs. physical equality difference for heap values.

Risk. **Medium**— Correct for integers (the common case) but differs for heap-allocated blocks.

16.2 NEQ

PDF Spec. NEQ: `accu = (accu != sp[0]); sp++`.

```
1042 Integer_comparison(intnat, NEQ, !=)
```

```
757 | NEQ =>
758   match s.(stack) with
759   | b :: rest => Step (st s pc' (if value_eqb s.(accu) b then
    val_false else val_true) rest s.(env) s.(extra_args)
    s.(global) s.(trap_sp))
760 | _ => Error "NEQ: stack underflow"
```

Analysis. Negation of EQ. Same caveats.

Status. **NEEDS TEST:** Same structural vs. physical equality issue.

Risk. **Medium**

16.3 LTINT

PDF Spec. LTINT: `accu = (accu < sp[0]); sp++`.

```
1043 Integer_comparison(intnat, LTINT, <)
```

```
763 | LTINT =>
764   match s.(accu), s.(stack) with
765   | Val_int a, Val_int b :: rest => Step (st s pc' (val_bool (a <?
    b)) rest s.(env) s.(extra_args) s.(global) s.(trap_sp))
766 | _, _ => Error "LTINT: type error or stack underflow"
```

Analysis. C: `(intnat)accu < (intnat)*sp++`. Works directly on tagged values because the tagging preserves order for signed integers. Rocq: `Z.ltb a b`. Exact match.

Status. **OK**

Risk. **Low**

16.4 LEINT

PDF Spec. LEINT: $\text{accu} = (\text{accu} \leq \text{sp}[0]); \text{sp}++$.

```
1044 Integer_comparison(intnat, LEINT, <=)
```

```
769 | LEINT ⇒  
770   match s.(accu), s.(stack) with  
771   | Val_int a, Val_int b :: rest ⇒ Step (st s pc' (val_bool (a  
    <=? b)) rest s.(env) s.(extra_args) s.(global) s.(trap_sp))  
772 | _, _ ⇒ Error "LEINT: type error or stack underflow"
```

Analysis. Same pattern. Z.leb. Exact match.

Status. **OK**

Risk. **Low**

16.5 GTINT

PDF Spec. GTINT: $\text{accu} = (\text{accu} > \text{sp}[0]); \text{sp}++$.

```
1045 Integer_comparison(intnat, GTINT, >)
```

```
775 | GTINT ⇒  
776   match s.(accu), s.(stack) with  
777   | Val_int a, Val_int b :: rest ⇒ Step (st s pc' (val_bool (a >  
    b)) rest s.(env) s.(extra_args) s.(global) s.(trap_sp))  
778 | _, _ ⇒ Error "GTINT: type error or stack underflow"
```

Analysis. Z.gtb. Exact match.

Status. **OK**

Risk. **Low**

16.6 GEINT

PDF Spec. GEINT: $\text{accu} = (\text{accu} \geq \text{sp}[0]); \text{sp}++$.

```
1046 Integer_comparison(intnat, GEINT, >=)
```

```
781 | GEINT ⇒  
782   match s.(accu), s.(stack) with  
783   | Val_int a, Val_int b :: rest ⇒ Step (st s pc' (val_bool (a  
784     >=? b)) rest s.(env) s.(extra_args) s.(global) s.(trap_sp))  
   | _, _ ⇒ Error "GEINT: type error or stack underflow"
```

Analysis. Z.geb. Exact match.

Status. OK

Risk. Low

16.7 OFFSETINT

PDF Spec. OFFSETINT n : $\text{accu} += n$.

```
1067 Instruct(OFFSETINT):  
1068   accu += *pc << 1;  
1069   pc++;  
1070   Next;
```

```
787 | OFFSETINT n ⇒  
788   match s.(accu) with  
789   | Val_int a ⇒ Step (st s pc' (Val_int (a + n)) s.(stack)  
790     s.(env) s.(extra_args) s.(global) s.(trap_sp))  
   | _ ⇒ Error "OFFSETINT: not an integer"
```

Analysis. C: $\text{accu} += *pc \ll 1$ (shift accounts for tagging). Rocq: $\text{Val_int}(a + n)$. Exact match.

Status. OK

Risk. Low

16.8 OFFSETREF

PDF Spec. OFFSETREF n : $\text{Field}(\text{accu}, 0) += n$; $\text{accu} = ()$. Increments/decrements the value stored in a reference.

```
1071 Instruct(OFFSETREF):  
1072   Field(accu, 0) += *pc << 1;  
1073   accu = Val_unit;  
1074   pc++;  
1075   Next;
```

```

793 | OFFSETREF n ⇒
794   match s.(accu) with
795   | Val_ptr addr ⇒
796     match heap_lookup s.(hp) addr with
797     | Some (_, Val_int old :: rest) ⇒
798       let new_hp      heap_update s.(hp) addr (Val_int (old + n) ::
799         rest) in
800       Step (mk_state pc' val_unit s.(stack) s.(env) s.(extra_args)
801         s.(global) s.(trap_sp) new_hp s.(next_addr))
801 | _ ⇒ Error "OFFSETREF: not a ref"

```

Analysis. C: `Field(accu, 0) += *pc << 1; accu = Val_unit`. Rocq: reads heap, adds n to first field, writes back via `heap_update`. Exact match.

Status. OK

Risk. Low

16.9 ISINT

PDF Spec. ISINT: `accu = Val_long(accu & 1)`. Tests if `accu` is an integer (tag bit set).

```

1076 Instruct(ISINT):
1077   accu = Val_long(accu & 1);
1078   Next;

```

```

805 | ISINT ⇒
806   Step (st s pc' (if is_int s.(accu) then val_true else val_false)
807     s.(stack) s.(env) s.(extra_args) s.(global) s.(trap_sp))
808   (* GETMETHOD: accu is the method index, stack top is the object.
809     Look up method at position accu in the object's class table. *)

```

Analysis. C: tests tag bit. Rocq: pattern matches on `Val_int`. Exact match.

Status. OK

Risk. Low

17 Unsigned Comparisons

17.1 ULTINT

PDF Spec. ULTINT: Unsigned less-than: `accu = ((uintnat)accu < (uintnat)sp[0])`.

```

1047 Integer_comparison(uintnat, ULTINT, <)

```

```

942 | ULTINT ⇒
943   match s.(accu), s.(stack) with
944   | Val_int a, Val_int b :: rest ⇒ Step (st s pc' (val_bool
      (Z.ltb (z_flip_sign a) (z_flip_sign b))) rest s.(env)
      s.(extra_args) s.(global) s.(trap_sp))
945   | _, _ ⇒ Error "ULTINT: type error or stack underflow"

```

Analysis. C uses `uintnat` cast for unsigned comparison on tagged values. Rocq uses `z_flip_sign` which XORs with 2^{62} (the sign bit) to convert signed to unsigned order. This is a standard trick: flipping the sign bit maps the signed range $[-2^{62}, 2^{62} - 1]$ to $[0, 2^{63} - 1]$ monotonically.

The C code compares *tagged* values as unsigned, while our code compares *untagged* values with sign-flip. These are equivalent because the tag bit (lowest bit, always 1) doesn't affect the ordering of tagged integers when both operands have the tag set.

Status. OK

Risk. Low

17.2 UGEINT

PDF Spec. UGEINT: Unsigned greater-or-equal.

```

1048 Integer_comparison(uintnat, UGEINT, >=)

```

```

948 | UGEINT ⇒
949   match s.(accu), s.(stack) with
950   | Val_int a, Val_int b :: rest ⇒ Step (st s pc' (val_bool
      (Z.geb (z_flip_sign a) (z_flip_sign b))) rest s.(env)
      s.(extra_args) s.(global) s.(trap_sp))
951   | _, _ ⇒ Error "UGEINT: type error or stack underflow"
952
953   (* BULTINT(n, target): branch if n < accu (unsigned). "n is ULT
      the integer accu."
954     BUG Eint(n, target): branch if n >= accu (unsigned). "n is UGE
      the integer accu."
955     Like signed B-ops (which use n CMP accu), both use z_flip_sign
      for unsigned order. *)

```

Analysis. Same technique as ULTINT with `Z.geb`. Exact match.

Status. OK

Risk. Low

18 Branch Comparisons

18.1 BEQ

PDF Spec. BEQ *n ofs*: If $n = \text{Long_val}(\text{accu})$, branch by *ofs*.

```
1058 Integer_branch_comparison(intnat, BEQ, ==, "==")
```

```
895 | BEQ n target =>
896   match s.(accu) with
897   | Val_int a => if Z.eqb a n then Step (st s target s.(accu)
898     s.(stack) s.(env) s.(extra_args) s.(global) s.(trap_sp))
899     else Step (st s pc' s.(accu) s.(stack) s.(env)
900       s.(extra_args) s.(global) s.(trap_sp))
901   (* Non-integer values are never equal to Val_int n, so don't
902     branch *)
903 | _ => Step (st s pc' s.(accu) s.(stack) s.(env) s.(extra_args)
904   s.(global) s.(trap_sp))
```

Analysis. C: if (*pc++ == Long_val(accu)) pc += *pc; else pc++. Rocq: matches Val_int a, checks Z.eqb a n. Non-integer accu falls through (no branch), which is correct since an integer literal can never equal a block.

Status. OK

Risk. Low

18.2 BNEQ

PDF Spec. BNEQ n ofs: Branch if $n \neq$ accu.

```
1059 Integer_branch_comparison(intnat, BNEQ, !=, "!=")
```

```
903 | BNEQ n target =>
904   match s.(accu) with
905   | Val_int a => if Z.eqb a n then Step (st s pc' s.(accu)
906     s.(stack) s.(env) s.(extra_args) s.(global) s.(trap_sp))
907     else Step (st s target s.(accu) s.(stack) s.(env)
908       s.(extra_args) s.(global) s.(trap_sp))
909   (* Non-integer values are never equal to Val_int n, so always
910     branch *)
911 | _ => Step (st s target s.(accu) s.(stack) s.(env)
912   s.(extra_args) s.(global) s.(trap_sp))
913
914 (* B-comparison instructions: semantics is *pc++ CMP
915   Long_val(accu),
916   i.e., the OPERAND is on the LEFT and ACCU on the RIGHT. *)
```

Analysis. Negation of BEQ. Non-integer accu always branches (correct: a block is never equal to an integer).

Status. OK

Risk. **Low**

18.3 BLTINT

PDF Spec. BLTINT n of s : Branch if $n < \text{accu}$ (the operand is on the left).

```
1060 Integer_branch_comparison(intnat, BLTINT, <, "<")
```

```
914 | BLTINT n target =>
915   match s.(accu) with
916   | Val_int a => if Z.ltb n a then Step (st s target s.(accu)
917     s.(stack) s.(env) s.(extra_args) s.(global) s.(trap_sp))
918     else Step (st s pc' s.(accu) s.(stack) s.(env)
919       s.(extra_args) s.(global) s.(trap_sp))
918 | _ => Error "BLTINT: not an integer"
```

Analysis. C macro: `if (*pc++ < (intnat)Long_val(accu))` — operand $< \text{accu}$. Rocq: `Z.ltb n a` — $n < a$. Exact match (note: n is the operand, a is `accu`).

Status. **OK**

Risk. **Low**

18.4 BLEINT

PDF Spec. BLEINT n of s : Branch if $n \leq \text{accu}$.

```
1061 Integer_branch_comparison(intnat, BLEINT, <=, "<=")
```

```
921 | BLEINT n target =>
922   match s.(accu) with
923   | Val_int a => if Z.leb n a then Step (st s target s.(accu)
924     s.(stack) s.(env) s.(extra_args) s.(global) s.(trap_sp))
925     else Step (st s pc' s.(accu) s.(stack) s.(env)
926       s.(extra_args) s.(global) s.(trap_sp))
925 | _ => Error "BLEINT: not an integer"
```

Analysis. `Z.leb n a`. Exact match.

Status. **OK**

Risk. **Low**

18.5 BGTINT

PDF Spec. BGTINT n of s : Branch if $n > \text{accu}$.


```
1062 Integer_branch_comparison(intnat, BGTINT, >, ">")
```

```
928 | BGTINT n target =>
929   match s.(accu) with
930   | Val_int a => if Z.gtb n a then Step (st s target s.(accu)
931     s.(stack) s.(env) s.(extra_args) s.(global) s.(trap_sp))
932     else Step (st s pc' s.(accu) s.(stack) s.(env)
933       s.(extra_args) s.(global) s.(trap_sp))
932 | _ => Error "BGTINT: not an integer"
```

Analysis. $Z.gtb\ n\ a$. Exact match.

Status. OK

Risk. Low

18.6 BGEINT

PDF Spec. BGEINT $n\ ofs$: Branch if $n \geq \text{accu}$.

```
1063 Integer_branch_comparison(intnat, BGEINT, >=, ">=")
```

```
935 | BGEINT n target =>
936   match s.(accu) with
937   | Val_int a => if Z.geb n a then Step (st s target s.(accu)
938     s.(stack) s.(env) s.(extra_args) s.(global) s.(trap_sp))
939     else Step (st s pc' s.(accu) s.(stack) s.(env)
940       s.(extra_args) s.(global) s.(trap_sp))
939 | _ => Error "BGEINT: not an integer"
```

Analysis. $Z.geb\ n\ a$. Exact match.

Status. OK

Risk. Low

18.7 BULTINT

PDF Spec. BULTINT $n\ ofs$: Unsigned branch if $n < \text{accu}$.

```
1064 Integer_branch_comparison(uintnat, BULTINT, <, "<")
```

```

957 | BULTINT n target ⇒
958   match s.(accu) with
959   | Val_int a ⇒ if Z.ltb (z_flip_sign n) (z_flip_sign a) then
      Step (st s target s.(accu) s.(stack) s.(env) s.(extra_args)
      s.(global) s.(trap_sp))
960           else Step (st s pc' s.(accu) s.(stack) s.(env)
      s.(extra_args) s.(global) s.(trap_sp))
961 | _ ⇒ Error "BULTINT: not an integer"

```

Analysis. C: (uintnat)*pc++ < (uintnat)Long_val(accu). Rocq: Z.ltb (z_flip_sign n) (z_flip_sign a). Same z_flip_sign technique as ULTINT. Exact match.

Status. OK

Risk. Low

18.8 BUGEINT

PDF Spec. BUGEINT *n ofs*: Unsigned branch if $n \geq \text{accu}$.

```

1065 Integer_branch_comparison(uintnat, BUGEINT, >=, ">=")

```

```

964 | BUGEINT n target ⇒
965   match s.(accu) with
966   | Val_int a ⇒ if Z.geb (z_flip_sign n) (z_flip_sign a) then
      Step (st s target s.(accu) s.(stack) s.(env) s.(extra_args)
      s.(global) s.(trap_sp))
967           else Step (st s pc' s.(accu) s.(stack) s.(env)
      s.(extra_args) s.(global) s.(trap_sp))
968 | _ ⇒ Error "BUGEINT: not an integer"

```

Analysis. Z.geb (z_flip_sign n) (z_flip_sign a). Exact match.

Status. OK

Risk. Low

19 Object-Oriented Operations

19.1 GETMETHOD

PDF Spec. GETMETHOD: accu = Field(Field(sp[0], 0), Int_val(accu)). Looks up a method in the object's class table.

```

1084 Instruct(GETMETHOD):
1085   accu = Lookup(sp[0], accu);
1086   Next;

```

```

810 | GETMETHOD ⇒
811   match s.(stack) with
812   | obj :: _ ⇒
813     match field_or_heap s obj 0 with
814     | Some class_tbl ⇒
815       match s.(accu) with
816       | Val_int n ⇒
817         match field_or_heap s class_tbl (Z.to_nat n) with
818         | Some method_fn ⇒ Step (st s pc' method_fn s.(stack)
819                               s.(env) s.(extra_args) s.(global) s.(trap_sp))
819         | None ⇒ Error "GETMETHOD: method not found"
820         | _ ⇒ Error "GETMETHOD: not an integer index"
821         | None ⇒ Error "GETMETHOD: no class table"
822     | _ ⇒ Error "GETMETHOD: stack underflow"
823
824   (* OCaml class table layout (from caml_get_public_method):
825      meths[0] = count (index of last+1 tag slot, always odd).
826      meths[1] = padding (0).
827      meths[2k] = closure of method k (even indices >= 2).
828      meths[2k+1] = tag of method k (odd indices >= 3).
829      The method for tag T is found by scanning pairs (closure, tag)
830                  starting
831                  after the first two header words. *)

```

Analysis. C: Lookup(sp[0], accu) = Field(Field(sp[0], 0), Int_val(accu)). Rocq: Gets field_or_heap obj 0 (class table), then field_or_heap class_tbl (Z.to_nat n). Exact match.

Status. OK

Risk. Low

19.2 GETPUBMET

PDF Spec. GETPUBMET tag: Push object, look up public method by tag. Uses binary search in the method table (with optional cache).

```

1126   Instruct (GETPUBMET):
1127     *--sp = accu;
1128     accu = Val_int(*pc);
1129     pc += 2;
1130     /* Fallthrough */
1131   #endif

```

```

835 | GETPUBMET tag ⇒
836   let new_stack s.(accu) :: s.(stack) in
837   match field_or_heap s s.(accu) 0 with
838   | Some class_tbl ⇒
839     (* collect all fields of class_tbl *)
840     let fields
841       match class_tbl with
842       | Val_block _ fs ⇒ fs

```

```

843 | Val_ptr addr ⇒ match heap_lookup s.(hp) addr with Some
      (_, fs) ⇒ fs | None ⇒ [] end
844 | _ ⇒ []
845 in
846   (* Linear scan: skip meths[0] (count) and meths[1] (padding),
847     then scan pairs (closure, tag). *)
848   let fix scan (remaining : list value) : step_result
849     match remaining with
850     | [] ⇒ Error "GETPUBMET: method not found"
851     | _ :: [] ⇒ Error "GETPUBMET: method not found"
852     | method_fn :: tag_val :: rest ⇒
853       if value_eqb tag_val (Val_int tag) then
854         Step (st s pc' method_fn new_stack s.(env)
              s.(extra_args) s.(global) s.(trap_sp))
855       else scan rest
856   in scan (skipn 2 fields)
857 | None ⇒ Error "GETPUBMET: no class table"
858
859 (* GETDYNMET: accu is method tag (val), stack top is object. *)

```

Analysis. C: Uses a cache (`*pc & Field(meths,1)`) for fast lookup, falling back to binary search: `li=3`, `hi=Field(meths,0)`, binary search on odd indices.

Rocq: Uses linear scan of pairs (`closure`, `tag`) starting after the 2-word header. This is semantically correct but slower.

Difference: Linear scan vs. binary search. Both produce the same result for well-formed method tables. The cache miss path in C also does binary search, and the Rocq linear scan covers the same elements.

The OCaml method table layout has:

- `meths[0]` = count/size
- `meths[1]` = cache mask
- Even indices ≥ 2 : closures
- Odd indices ≥ 3 : tags

Our scan skips 2 header words and processes pairs, which matches the (`closure`, `tag`) layout at even/odd positions.

Status. **NEEDS TEST:** Linear scan correctness for OO programs.

Risk. **Medium**— Different algorithm but same result; needs testing with OO code.

19.3 GETDYNMET

PDF Spec. GETDYNMET: Dynamic method lookup. `accu` is the method tag, `sp[0]` is the object.

```

1132 Instruct(GETDYNMET): {
1133   /* accu == tag, sp[0] == object, *pc == cache */
1134   value meths = Field(sp[0], 0);
1135   int li = 3, hi = Field(meths,0), mi;
1136   while (li < hi) {
1137     mi = ((li+hi) >> 1) | 1;
1138     if (accu < Field(meths,mi)) hi = mi-2;

```

```

1139     else li = mi;
1140   }
1141   accu = Field (meths, li-1);
1142   Next;

863 | GETDYNMET ⇒
864   match s.(stack) with
865   | obj :: _ ⇒
866     let tag = s.(accu) in
867     match field_or_heap s obj 0 with
868     | Some class_tbl ⇒
869       let fields
870         match class_tbl with
871         | Val_block _ fs ⇒ fs
872         | Val_ptr addr ⇒ match heap_lookup s.(hp) addr with Some
873           (_, fs) ⇒ fs | None ⇒ [] end
874       in
875         (* Linear scan: skip meths[0] (count) and meths[1] (padding),
876            then scan pairs (closure, tag). *)
877         let fix scan (remaining : list value) : step_result
878           match remaining with
879           | [] ⇒ Error "GETDYNMET: method not found"
880           | _ :: [] ⇒ Error "GETDYNMET: method not found"
881           | method_fn :: tag_val :: rest ⇒
882             if value_eqb tag_val tag then
883               Step (st s pc' method_fn s.(stack) s.(env)
884                 s.(extra_args) s.(global) s.(trap_sp))
885             else scan rest
886         in scan (skipn 2 fields)
887         | None ⇒ Error "GETDYNMET: no class table"
888         | _ ⇒ Error "GETDYNMET: stack underflow"
889         (* B-comparison instructions: the spec says "increments pc by
890            ofs-1 if val CMP accu".
891            In interp.c the operand is an absolute instruction index (after
            decode). *)

```

Analysis. Same as GETPUBMET but without cache. C uses binary search, Rocq uses linear scan. Same correctness argument.

Status. **NEEDS TEST:** Same as GETPUBMET.

Risk. **Medium**

20 Miscellaneous

20.1 STOP

PDF Spec. STOP: Terminate execution, return accu.

```

1147   Instruct(STOP):
1148       Caml_state->external_raise = initial_external_raise;
1149       Caml_state->extern_sp = sp;
1150       caml_callback_depth--;
1151       return accu;

```

```

971 | STOP ⇒ Halt s.(accu)

```

Analysis. C: restores initial state, returns `accu`. Rocq: returns `Halt accu`. Exact match.

Status. OK

Risk. Low

20.2 EVENT

PDF Spec. EVENT: Debugger event hook.

```

1153   Instruct(EVENT):
1154       if (--caml_event_count == 0) {
1155           Setup_for_debugger;
1156           caml_debugger(EVENT_COUNT, Val_unit);
1157           Restore_after_debugger;
1158       }
1159       Restart_curr_instr;

```

```

972 | EVENT ⇒ Step (st s pc' s.(accu) s.(stack) s.(env)
    s.(extra_args) s.(global) s.(trap_sp))

```

Analysis. Modeled as no-op (advance PC). In C, triggers debugger if event count reaches zero. Safe to skip since we don't model the debugger.

Status. OK

Risk. Low

20.3 BREAK

PDF Spec. BREAK: Debugger breakpoint.

```

973 | BREAK ⇒ Step (st s pc' s.(accu) s.(stack) s.(env)
    s.(extra_args) s.(global) s.(trap_sp))

```

Analysis. Modeled as no-op, same as EVENT.

Status. **OK**

Risk. **Low**

20.4 **PERFORM**

PDF Spec. OCaml 5.x effect handlers. Not present in OCaml 4.14.

```
975 | PERFORM ⇒ Error "PERFORM: effects not supported"
```

Analysis. Returns `Error "PERFORM: effects not supported"`. Correct for OCaml 4.14 (these opcodes don't exist). If targeting OCaml 5.x, these would need implementation.

Status. **STUB**

Risk. **Low**— Not used by OCaml 4.14 bytecode.

20.5 **RESUME**

Analysis. Same as **PERFORM**. Returns error.

Status. **STUB**

Risk. **Low**

20.6 **RESUMETERM**

Analysis. Same as **PERFORM**. Returns error.

Status. **STUB**

Risk. **Low**

20.7 **REPERFORMTERM**

Analysis. Same as **PERFORM**. Returns error.

Status. **STUB**

Risk. **Low**

21 Findings Summary

21.1 Statistics

Category	Count
Total instructions audited	100
OK	82
Needs test	11
Bug found	1
Stub (not implemented)	4
Effect handlers (N/A for 4.14)	4

21.2 Summary Table

	Instruction	Status	Risk	Notes
	ACC	OK	Low	
	PUSH	OK	Low	
	PUSHACC	OK	Low	
	POP	OK	Low	
	ASSIGN	OK	Low	
	ENVACC	OK	Low	
	PUSHENVACC	OK	Low	
	OFFSETCLOSURE	OK	Low	
	PUSHOFFSETCLOSURE	OK	Low	
	PUSH_RETADDR	OK	Low	
	APPLY	OK	Low	
	APPLY1	OK	Low	
	APPLY2	OK	Low	
	APPLY3	OK	Low	
	APPTERM	OK	Low	
	APPTERM1	OK	Low	
	APPTERM2	OK	Low	
	APPTERM3	OK	Low	
	RETURN	OK	Low	
	RESTART	OK	Medium	Closure offset arithmetic
	GRAB	Needs test	High: closinfo	closinfo=0, pc offset
	CLOSURE	OK	Medium	closinfo=0
	CLOSUREREC	Needs test	High: complex	Infix hdrs, closinfo, push order
	GETGLOBAL	OK	Low	
	PUSHGETGLOBAL	OK	Low	
	GETGLOBALFIELD	OK	Low	
	PUSHGETGLOBALFIELD	OK	Low	
	SETGLOBAL	OK	Low	
	ATOM	OK	Low	
	PUSHATOM	OK	Low	
	MAKEBLOCK	OK	Low	
	MAKEBLOCK1	OK	Low	
	MAKEBLOCK2	OK	Low	

	Instruction	Status	Risk	Notes
MAKEBLOCK3	OK	Low		
MAKEFLOATBLOCK	Needs test	Medium		Float model
GETFIELD	OK	Low		
GETFLOATFIELD	Needs test	Medium		Float model
SETFIELD	OK	Low		
SETFLOATFIELD	BUG: inverted	High: inverted ops		accu/sp swapped
VECTLENGTH	OK	Low		
GETVECTITEM	OK	Low		
SETVECTITEM	OK	Low		
GETSTRINGCHAR	OK	Low		
GETBYTESCHAR	OK	Low		
SETBYTESCHAR	OK	Low		
BRANCH	OK	Low		
BRANCHIF	OK	Low		
BRANCHIFNOT	OK	Low		
SWITCH	OK	Low		Depends on loader
BOOLNOT	OK	Low		
PUSHTRAP	OK	Medium		Trap link repr differs
POPTRAP	OK	Low		
RAISE	OK	Medium		Trap navigation
RERAISE	OK	Low		
RAISE_NOTRACE	OK	Low		
CHECK_SIGNALS	OK	Low		No-op
C_CALL	OK	Low		Interaction tree
CONSTINT	OK	Low		
PUSHCONSTINT	OK	Low		
NEGINT	OK	Low		
ADDINT	OK	Low		
SUBINT	OK	Low		
MULINT	OK	Low		
DIVINT	OK	Low		Raises on /0
MODINT	OK	Low		Raises on /0
ANDINT	OK	Low		
ORINT	OK	Low		
XORINT	OK	Low		
LSLINT	OK	Low		
LSRINT	OK	Low		
ASRINT	OK	Low		
EQ	Needs test	Medium		Structural vs physical
NEQ	Needs test	Medium		Structural vs physical
LTINT	OK	Low		
LEINT	OK	Low		
GTINT	OK	Low		
GEINT	OK	Low		
OFFSETINT	OK	Low		
OFFSETREF	OK	Low		
ISINT	OK	Low		

	Instruction	Status	Risk	Notes
ULTINT	OK	Low		
UGEINT	OK	Low		
BEQ	OK	Low		
BNEQ	OK	Low		
BLTINT	OK	Low		
BLEINT	OK	Low		
BGTINT	OK	Low		
BGEINT	OK	Low		
BULTINT	OK	Low		
BUGEINT	OK	Low		
GETMETHOD	OK	Low		
GETPUBMET	Needs test	Medium		Linear scan vs binary search
GETDYNMET	Needs test	Medium		Linear scan vs binary search
STOP	OK	Low		
EVENT	OK	Low		No-op
BREAK	OK	Low		No-op
PERFORM	STUB	Low		OCaml 5.x only
RESUME	STUB	Low		OCaml 5.x only
RESUMETERM	STUB	Low		OCaml 5.x only
REPERFORMTERM	STUB	Low		OCaml 5.x only

22 Known Discrepancies and Action Items

22.1 Bug: SETFLOATFIELD operand inversion

Severity: High. In `interp.c`, `SETFLOATFIELD` treats `accu` as the float array and `*sp` as the new value. Our Rocq code swaps these: it treats the stack top as the array and `accu` as the new value.

Action: Fix `Interp.v` to match the C convention. Specifically, `SETFLOATFIELD n` should:

1. Read field n from `accu` (the float array)
2. Pop new value from stack
3. Store new value into field n
4. Set `accu = Val_unit`

Impact: Affects programs using mutable float arrays (`Array.Floatarray`). Most OCaml programs don't use these directly, but the standard library does internally.

22.2 Systemic: closinfo field hardcoded to 0

Severity: Medium. Affects: `GRAB`, `CLOSURE`, `CLOSUREREC`.

The `closinfo` field in closure blocks is hardcoded to `Val_int 0` instead of the proper `Make_closinfo(arity, start_env)` value. In `interp.c`, this field encodes:

- The arity of the function (bits 56–63 on 64-bit)
- The start-of-environment offset (bits 0–55)

Currently safe because nothing in our interpreter inspects `closinfo`. Would become a bug if we implement:

- Marshaling/serialization (reads closinfo to find env offset)
- The `caml_apply*` C primitives (read arity)
- Debugger integration

Action: Compute the proper closinfo value. Low priority until one of the above features is needed.

22.3 Simplification: structural vs. physical equality

Severity: Medium. Affects: EQ, NEQ.

Our `value_eqb` performs structural equality while `interp.c` performs physical (pointer) equality. For immutable values this is equivalent, but for mutable heap-allocated values, physical equality can distinguish aliased references from structurally equal copies.

Action: For `Val_ptr` values, compare addresses only (not heap contents). This is a one-line fix in `value_eqb`.

22.4 Simplification: GETPUBMET/GETDYNMET linear scan

Severity: Low. Our linear scan produces correct results but is $O(n)$ per method lookup instead of $O(\log n)$. Not a correctness issue, only a performance concern for extracted code running OO-heavy programs.

Action: Implement binary search to match `interp.c`. Low priority.

22.5 Simplification: float representation

Severity: Low–Medium. Affects: MAKEFLOATBLOCK, GETFLOATFIELD, SETFLOATFIELD, VECTLENGTH.

Our model stores floats as regular values (no flat/boxed distinction). The C interpreter uses `Double_wosize` words per float and has special flat storage. This doesn't affect correctness for integer-only programs since float operations go through C primitives.

Action: Revisit when float support is prioritized.

22.6 Note: infix headers in CLOSUREREC

Severity: Low. Our infix headers are `Val_block Infix_tag []` (empty block) while the C version uses `Make_header(i*3, Infix_tag, Caml_white)` (a proper GC header encoding the offset). Since we have no GC and track offsets via `Val_closure`, this is harmless. Would need revision if we ever model GC scanning.